UltraGS: руководство программиста

UltraGS - звуковая карта для Sprinter, один из режимов работы платы PicoGUS. Это руководство объясняет, как программа на Z80 загружает в карту звуки, запускает музыку и эффекты и следит за состоянием карты. Оно рассчитано на программиста, который читает ассемблер Z80 (`LD`, `CALL`, `RET`, переходы) и пишет для DSS. Знакомство с General Sound полезно, но не требуется.

Как читать. Глава 1 вводит понятия, без которых остальное непонятно: объект, запуск, голос, два интерфейса карты. Глава 2 - подробно разобранная первая программа: она собирается и работает. Главы 3-8 описывают расширенный API UltraGS: обмен, память, воспроизведение, потоковую подачу и наблюдение за картой. Глава 9 посвящена командам General Sound, глава 10 - синтезатору OPL2. Приложения содержат справочные таблицы: порты, коды ошибок, точные форматы всех команд. К ним обращаются во время написания своей программы.

Все листинги взяты из собираемых примеров в каталоге `examples/`; как их собрать и запустить, описано в README. Текущая версия интерфейса - API 1.0 / VLS1. Редакция 8, 28 октября 2026 года.

Оглавление

1. Карта глазами программиста 4
   1.1. Что умеет карта . . . 4
   1.2. Объекты, запуски и голоса . . . 4
   1.3. Два интерфейса . . . 5
   1.4. Библиотека для Z80 . . . 5
2. Первый звук: низкий тон из собственного массива 8
   2.1. Обратиться к нужной карте . . . 8
   2.2. Подготовить числа, которые станут звуком . . . 8
   2.3. Выделить память . . . 9
   2.4. Скопировать массив на карту . . . 10
   2.5. Сообщить карте, как читать массив . . . 10
   2.6. Включить звук . . . 11
   2.7. Остановить звук и вернуть память . . . 12
   2.8. Дополнительная проверка: прочитать свои байты обратно . . . 12
   2.9. Вся последовательность и первые изменения . . . 13
3. Обмен с картой 15
   3.1. Запрос и ответ . . . 15
   3.2. Передача данных блоками . . . 16
   3.3. Ошибки и потеря ответа . . . 16
   3.4. Сеанс . . . 17
   3.5. Исходное состояние карты . . . 17
4. Объекты и память 19
   4.1. Жизненный цикл объекта . . . 19
   4.2. Что сообщить карте о данных . . . 19
   4.3. Что можно узнать об объекте . . . 20
   4.4. Что освобождать при выходе . . . 20
5. Воспроизведение 21
   5.1. Основной звук . . . 21
   5.2. Эффекты . . . 21
   5.3. Громкость, панорама и запас . . . 21
   5.4. Голоса . . . 22
6. Пример: музыка и эффект поверх неё 23
   6.1. Загрузка файла блоками . . . 23
   6.2. Запуск музыки и эффекта . . . 24
   6.3. Очистка . . . 26
7. Поток с диска 28
   7.1. Когда нужен поток . . . 28
   7.2. Команды потока . . . 28
   7.3. Пример: UGPLAY STREAM . . . 28
   7.4. Ограничение . . . 29
8. Наблюдение за картой 30
   8.1. Снимок состояния . . . 30
   8.2. Голоса и эффекты постранично . . . 30
   8.3. Счётчики . . . 30
9. Команды General Sound 31
   9.1. Как устроен обмен GS . . . 31
   9.2. Таблица команд . . . 32
   9.3. Загрузка и управление модулем . . . 35
   9.4. Эффекты GS . . . 36
   9.5. Отличия от General Sound и NeoGS . . . 37
10. AdLib OPL2 38

10.1. Доступ к синтезатору . . . 38
10.2. Инструмент и нота . . . 39
11. Приложения 41
11.1. Порты, биты состояния и возможностей . . . 41
11.2. Коды ошибок . . . 43
11.3. Утилиты . . . 44
11.4. Словарь . . . 44
11.5. Источники . . . 44
11.6. Форматы команд UltraGS API 1.0 . . . 45
11.7. Вложенные структуры и записи . . . 61

1. Карта глазами программиста

1.1. Что умеет карта

Карта сама формирует звук из данных, которые ей передала программа. После команды воспроизведения Z80 свободен: он может рисовать экран, читать клавиатуру или грузить следующий файл, а музыка продолжает играть.

Вид звука	Что передаёт программа	Для чего удобно
RAW PCM	Массив отсчётов; частоту и формат сообщает отдельной командой	Короткие эффекты, собственный генератор звука
WAV PCM	Файл с заголовком; частоту и формат карта берёт из него	Записанная речь, звуки, музыка
WAV IMA ADPCM	Файл WAV со сжатием IMA ADPCM (кодек 0x11); карта распаковывает его при воспроизведении	То же при вчетверо меньшем объёме данных
MOD, S3M	Модуль трекера: инструменты и ноты	Фоновая музыка; ноты разбирает карта, а не Z80
OPL2	Настройки регистров синтезатора и нажатия клавиш	Синтез без загрузки отсчётов, совместимость с AdLib

Отсчёт - одно число, описывающее мгновенное значение звуковой волны; **PCM** - запись звука последовательностью отсчётов. Карта принимает отсчёты 8 или 16 бит, mono или stereo, с частотой от 8000 до 48000 в секунду, а на выход всегда выдаёт 48000 Гц stereo 16 бит. Файлы WAV со сжатием IMA ADPCM (4 бита на отсчёт) карта распаковывает сама.

Для хранения звуков на плате есть 8 МиБ памяти. Она не видна в адресном пространстве Z80: записать или прочитать её можно только командами карты. Загруженные данные остаются в ней, пока программа их не удалит или карту не сбросят.

Одновременно карта смешивает до 32 **голосов**. Голос - один канал воспроизведения PCM: mono-эффект занимает один голос, stereo-эффект - два, четырёхканальный MOD - четыре, восьмиканальный - восемь. Синтезатор OPL2 работает отдельно и голоса не расходует.

1.2. Объекты, запуски и голоса

У каждого звука на карте три разных номера, и их важно не путать.

Объект - это данные, хранящиеся в памяти карты: массив отсчётов, файл WAV или MOD. Карта выдаёт объекту номер - **handle**. Handle не является адресом памяти и не выбирается программистом: его возвращает команда ALLOC, и его нужно сохранить.

Запуск (instance) - отдельное воспроизведение объекта. Один и тот же объект можно запустить несколько раз; каждый запуск получает свой номер - **play_id** для основного звука или **instance_id** для эффекта. Останавливают запуск, а удаляют объект.

Голос - физический ресурс микшера, который карта назначает запуску сама. Программа обычно о голосах не думает; номера голосов нужны только при чтении состояния карты.

Пример. Программа загрузила stereo WAV и получила handle 17. Она запустила его дважды как эффект и получила номера запусков 5 и 6. Данные хранятся один раз, но голосов занято четыре: по два на каждый stereo-запуск. STOP_FX(5) остановит только первый запуск. FREE(17) сразу после этого вернёт ошибку PINNED: объект ещё используется вторым запуском. После STOP_FX(6) объект можно удалить. Числа здесь условные: реальные значения всегда берут из ответов карты.

Основной звук один: MOD, S3M, PCM или поток. Он называется **foreground**. Независимые эффекты (FX) играют поверх него, и остановка музыки их не затрагивает. Новый PLAY основного звука заменяет предыдущий.

1.3. Два интерфейса

Для новой программы используйте **расширенный API UltraGS**. Это набор команд с номерами и аргументами фиксированного формата: выделить память, загрузить, запустить, приостановить, прочитать состояние. Именно его показывает глава 2, и для него есть готовая библиотека Z80.

Команды General Sound могут использоваться для лёгкого портирования программ, уже написанных для GS: они загружают модуль побайтно, управляют им однобайтовыми командами и читают флаги. UltraGS исполняет музыкальные команды GS 1.04 с дополнениями 1.05 (глава 9), но не эмулирует саму плату: внутреннего Z80 и его RAM нет, команды POKE/PEEK/ CALL не выполняются, MP3-декодер и SD-карта NeoGS не предоставляются.

Оба интерфейса работают с одной памятью и одним основным звуком. Музыка, запущенная командами GS, видна расширенному API и наоборот.

1.4. Библиотека для Z80

Библиотека `ugproto.asm` берёт на себя обмен с портами карты. Программа вызывает несколько процедур:

Процедура	Регистры на входе	Назначение
`UG_LIB_INIT`	-	Обнулить переменные библиотеки; вызывается первой
`UG_OPEN`	A = слот (0 или 1), HL = BASE	Найти карту, проверить режим UltraGS, запомнить адрес
`UG_RPC`	A = код команды, HL = аргументы, BC = их длина, DE = буфер ответа, IX = его размер	Выполнить команду расширенного API
`UG_WRITE` / `UG_READ`	HL = handle, DE = буфер Z80, BC = длина 1..1024, A = способ передачи, `UG_OFFSET` = смещение	Передать блок данных на карту или с карты
`UG_GS_COMMAND`	HL = 9-байтовое описание команды	Выполнить команду General Sound
`UG_GS_SD` / `UG_GS_WD`	A = байт (для `UG_GS_SD`)	Отдельные шаги обмена GS: записать байт в DATA и дождаться, пока карта его заберёт (9.1)
`UG_ISA_OPEN` / `UG_ISA_CLOSE`	A = слот, HL = адрес порта ISA (для `UG_ISA_OPEN`)	Открыть окно ISA для прямого обращения к портам, например OPL2 (10.1), и закрыть его
`UG_SNAPSHOT`	HL = буфер 128 байт	Получить снимок состояния (то же, что команда SNAPSHOT)
`UG_XCTRL`	A = управляющее значение	Служебные действия: сброс обмена, отмена передачи
`UG_FRAME_CRC`	A = 0 или 1	Включить контрольную сумму команд и ответов
`UG_CLOSE`	-	Завершить работу с картой

Соглашения одинаковы для всех процедур. После вызова установленный флаг переноса (CF=1) означает ошибку, её код находится в A; при успехе CF=0. `UG_RPC` дополнительно возвращает в BC фактическую длину ответа - её обязательно сравнивают с ожидаемой. Регистры IX и IY сохраняются; AF, BC, DE, HL и альтернативный набор могут измениться. Библиотека не рассчитана на вызов из обработчика прерываний.

Обращаясь к карте, библиотека на короткое время отображает порты ISA в область памяти C000h..FFFFh и закрывает это окно перед возвратом. Поэтому код, стек и буферы программы должны лежать ниже C000h. Закрывая окно, библиотека возвращает прежние PAGE3 и флаг прерываний, а в порт SYSTEM (1FFDh) пишет стандартное для DSS значение 01h: прочитать прежнее нельзя, порт только для записи. Окна 0-2

библиотека не трогает. Между вызовами окно закрыто, и программа может свободно вызывать DSS. Длинные передачи библиотека сама разбивает на короткие порции, так что прерывания не блокируются надолго.

Числовые коды команд, размеры аргументов и ответов, флаги и коды ошибок определены в генерируемом файле `ug_wire.inc`. В примерах встречаются имена `UG_OP_…` (код команды), `UG_OP_…_REQ_LEN` и `UG_OP_…_RESP_LEN` (размеры аргументов и ответа), `UG_ERR_…` (коды ошибок), `UG_KIND_…` (виды объектов), `UG_TRANSPORT_…` (способы передачи), `UG_MOD_…` (флаги PLAY_MOD), `UG_VOICE_AUTO` (выбор голоса картой), `UG_XC_…` (действия `UG_XCTRL`) и `UG_GS_GS_…` (коды команд General Sound). В коде лучше писать имя, а не число.

Варианты сборки

Перед `INCLUDE "ugproto.asm"` программа задаёт определения, от которых зависит, как библиотека ждёт ответа карты и как обращается с частотой процессора. Примеры руководства собраны с `UG_NO_DSS`; сами программы при этом вызывают DSS для чтения часов и клавиатуры.

Определение	Что делает библиотека
`UG_NO_DSS`	Не вызывает DSS. Срок ожидания отсчитывает числом опросов карты, ESC сама не проверяет
`UG_DSS_TIMED`	Отмеряет сроки по часам DSS; ESC прерывает долгое ожидание с кодом CANCELLED (15h)
`UG_ISA_TURBO_OFF`	Выключает Turbo на время каждого обращения к карте, после него включает

Из `UG_NO_DSS` и `UG_DSS_TIMED` выбирают одно. `UG_ISA_TURBO_OFF` сочетается с любым из них и нужен редко, см. ниже.

Без DSS. Такая сборка полезна, если программа должна исключить обращения к DSS, например потому что сама распоряжается памятью и прерываниями. Сроки рассчитаны на 21 МГц и на медленном процессоре только удлиняются: обычная команда ждёт ответа не меньше секунды, SEAL - не меньше 30 с. Чтобы долгое ожидание можно было прервать, программа определяет процедуру `UG_POLL_HOOK`. Библиотека вызывает её между опросами карты при закрытом окне ISA; CF=1 на выходе прерывает ожидание, и вызов библиотеки возвращает CANCELLED. Процедура может менять любые регистры, кроме SP:

```
    DEFINE UG_NO_DSS
    DEFINE UG_POLL_HOOK game_poll   ; необязательно
...
game_poll:                          ; CF=1 - прервать ожидание карты
    ld a,(quit_request)             ; 1, если обработчик клавиатуры игры
    rra                             ; заметил выход: бит 0 уходит в CF
    ret
```

Частота процессора. Библиотека не меняет частоту: обмен с картой идёт в том режиме, который выбрала программа, обычно в Turbo. Порты карты на это рассчитаны, кроме двух случаев:

Порты	Ограничение
1D0h-1D2h, настройка карты	Доступны только при выключенном Turbo. Библиотека их не использует: `UG_OPEN` узнаёт карту по её регистрам и команде INFO. Настройки меняет `UGCFG`
388h, состояние OPL2	Чтение надёжно в Turbo, пока в настройках карты не включён дополнительный блок портов BULK alias вне 380h-39Fh (по умолчанию он выключен). Запись регистров ограничения не имеет

Из-за первого случая `UG_OPEN` должен заранее знать адрес BASE: по умолчанию 02A0h, его задаёт `UGCFG`. Карта в другом режиме для него выглядит как отсутствующая: он возвращает TIMEOUT.

Если на конкретном компьютере обмен по ISA при включённом Turbo нестабилен, соберите программу с `UG_ISA_TURBO_OFF`. Тогда библиотека проводит каждое обращение к карте на пониженной частоте, а по его окончании включает Turbo, как принято в DSS. Между вызовами программа работает на обычной скорости, но сами передачи данных становятся медленнее. В этой сборке `UG_OPEN` узнаёт карту через

порты настройки, поэтому неверный режим карты даёт STATE.

2. Первый звук: низкий тон из собственного массива

Задача: получить короткий гудок без WAV-файла и без музыкального модуля. Программа создаст массив из 256 байт, загрузит его в карту, будет повторять примерно 1-2 секунды, затем остановит звук и освободит память. ESC позволяет закончить раньше. При успешном выходе загруженный массив на карте не остаётся.

Все фрагменты ниже взяты из firstpcm.asm. Они являются частями одной программы, а не отдельными EXE. Библиотечные процедуры начинаются с `UG_`; остальные, например `upload_sound`, написаны в самом примере и разобраны ниже. Полный файл содержит также заголовок EXE, общий выход и обработку ошибок.

Карта должна быть переведена в режим UltraGS утилитой `UGCFG`. Перед работой программа приводит карту в исходное состояние: останавливает звук и удаляет объекты, оставшиеся от предыдущих программ (3.5). Других программ, управляющих картой одновременно с этой, быть не должно.

2.1. Обратиться к нужной карте

Сначала библиотеке нужно сообщить, **где находится карта**. Здесь два независимых параметра:

Параметр	Значение в примере	Что оно означает
Номер слота	`DOC_SLOT`, по умолчанию 0	В какой ISA-слот Sprinter установлена карта: 0 или 1
Базовый адрес портов	`CARD_IO_BASE`, равный `02A0h`	С какого номера начинаются порты UltraGS на шине ISA

`DOC_SLOT` задаётся при сборке ключом ассемблера `-DDOC_SLOT=0` или `-DDOC_SLOT=1`; без ключа используется 0. Так из одного исходника получаются `PCM0.EXE` для первого слота и `PCM1.EXE` для второго.

Порт - адрес, по которому компьютер обращается к регистру устройства: передаёт команду или читает ответ. Карта занимает 32 соседних порта; адрес первого из них называют **базовым**, или BASE. `02A0h` - стандартная настройка UltraGS. Текущий адрес показывает `UGCFG INFO`, а `UGCFG PORTS` позволяет его изменить. `UG_OPEN` только использует уже установленный адрес, но не меняет настройки карты.

```
CARD_IO_BASE EQU 0x02A0       ; первый порт UltraGS на шине ISA
```

Далее передаём библиотеке номер слота через регистр A, а адрес портов через пару HL:

```
    call UG_LIB_INIT          ; подготовить переменные библиотеки
    ld a,DOC_SLOT             ; слот с картой: 0 или 1
    ld hl,CARD_IO_BASE        ; адрес первого порта карты
    call UG_OPEN              ; проверить, что карта отвечает
    ret c                     ; при ошибке вернуться к общему выходу
```

`UG_LIB_INIT` подготавливает переменные библиотеки в памяти Z80. `UG_OPEN` проверяет ответ карты и запоминает, куда обращаться при следующих вызовах. **Звука после этого ещё нет:** массив пока не загружен и команда воспроизведения не дана.

`RET C` после каждого вызова означает: при ошибке вернуться к общему выходу программы, не продолжая работу с картой. Код ошибки остаётся в A.

В полном примере после открытия также проверяются версия карты, наличие нужных функций, затем сброс карты в исходное состояние. Эти служебные шаги выполняет `check_card`; её текст разобран в 3.1 и 3.5.

2.2. Подготовить числа, которые станут звуком

Выберем простой формат **U8 mono**. U8 означает, что один отсчёт занимает один беззнаковый байт со значением 0..255. Середина этого диапазона - 128. Массив из одних значений 128 даёт тишину. Если значения периодически меняются выше и ниже середины, получается тон.

Наш массив содержит 32 значения 144, затем 32 значения 112. Это один период волны из 64 отсчётов. Повторим его четыре раза и получим 256 байт. `DUP/EDUP` в sjasmplus повторяют указанную группу при

сборке; DB записывает байт в программу. Здесь ещё ничего не передаётся карте.

```
samples:
    DUP 4                   ; четыре одинаковых периода волны
        DUP 32
            DB 144          ; 32 отсчёта выше середины
        EDUP
        DUP 32
            DB 112          ; 32 отсчёта ниже середины
        EDUP
    EDUP
    ASSERT $-samples == TONE_BYTES ; 4 * (32 + 32) = 256 байт
```

Читать эти числа карта будет с частотой 8000 отсчётов в секунду. Один период занимает 64 отсчёта, поэтому основная частота гудка будет **8000 / 64 = 125 Гц**. Весь массив длится только 256 / 8000 = 0,032 секунды; чтобы услышать длинный гудок, позднее включим повтор массива до команды остановки.

Настройки примера вынесены в начало исходника. Их названия используются дальше вместо чисел, смысл которых пришлось бы вспоминать:

```
TONE_RATE   EQU 8000        ; 8000 отсчётов звука в секунду
TONE_BYTES  EQU 256         ; длина нашего массива в байтах
TONE_GAIN   EQU 16          ; громкость: 0 = тишина, 64 = максимум
PAN_CENTER  EQU 128         ; по центру между левым и правым выходами
PCM_U8      EQU 0           ; формат: один беззнаковый байт на отсчёт
```

2.3. Выделить память

Команда **ALLOC** создаёт место для нового звука в памяти карты. Ей нужно передать вид данных и размер. Для этого подготовим небольшой блок аргументов в памяти Z80:

```
alloc_request:
    DB UG_KIND_RAW_PCM      ; будут обычные отсчёты, без заголовка файла
    DD TONE_BYTES           ; нужно 256 байт памяти на карте
```

`UG_KIND_RAW_PCM` означает массив отсчётов без файлового заголовка. `DD` записывает четырёхбайтовое число. Поэтому аргументы занимают **1 + 4 = 5 байт**. Сами 256 звуковых байт в эти пять байт не входят: их передадим следующим шагом.

Отправка команды выглядит так:

```
    ld a,UG_OP_ALLOC        ; команда: выделить память
    ld hl,alloc_request     ; адрес её аргументов в памяти Z80
    ld bc,UG_OP_ALLOC_REQ_LEN ; размер аргументов: 5 байт
    ld ix,UG_OP_ALLOC_RESP_LEN ; ожидаем ответ из 6 байт
    call rpc                ; отправить команду, получить response
    ret c                   ; ошибка: следующий шаг не выполнять
    ld hl,(response)        ; первые 2 байта: номер нового объекта
    ld (sound_handle),hl    ; запомнить этот номер
```

Здесь `rpc` - небольшая процедура нашего примера: она подставляет адрес буфера ответа, вызывает `UG_RPC` и проверяет, что длина ответа совпала с ожидаемой. У всех её вызовов один и тот же смысл параметров:

Регистр	Что передаём
A	Какую команду выполнить, например `UG_OP_ALLOC`
HL	Где в памяти Z80 лежат её аргументы
BC	Сколько байт занимают аргументы
IX	Сколько байт данных должно прийти в ответ

Вот она целиком. `response` - буфер на 128 байт в памяти Z80, куда `UG_RPC` кладёт данные ответа:

```
; A = команда, HL = аргументы, BC = их длина, IX = ожидаемая длина ответа.
; Ответ попадает в response; другая длина ответа считается ошибкой LENGTH.
rpc:
    ld de,response          ; буфер ответа, 128 байт
    call UG_RPC
    ret c
    push ix
    pop hl
    or a
    sbc hl,bc               ; длина ответа должна совпасть с ожидаемой
    ret z
    ld a,UG_ERR_LENGTH
    scf
    ret
```

За именем `UG_OP_ALLOC` стоит код команды 10h. Для ALLOC ответ состоит из двухбайтового номера объекта и четырёхбайтового фактического размера выделенной памяти: всего шесть байт.

`rpc` помещает ответ в массив `response`. Команда `LD HL,(response)` читает из его начала двухбайтовый **номер выделенного объекта**. Сохраняем номер в переменной `sound_handle`. Например, карта может вернуть 5; тогда дальнейшие операции с этим звуком используют 5. В другой раз номер может быть другим. Вручную подставлять постоянный номер нельзя.

2.4. Скопировать массив на карту

Место выделено, но звуковых данных в нём пока нет. Передадим массив `samples` из памяти Z80 в объект с полученным номером:

```
upload_sound:
    ld hl,0
    ld (UG_OFFSET),hl       ; смещение = 0: пишем с начала объекта
    ld (UG_OFFSET+2),hl     ; смещение занимает 4 байта, обнуляем оба слова
    ld hl,(sound_handle)    ; куда: объект в памяти карты
    ld de,samples           ; откуда: массив в памяти Z80
    ld bc,TONE_BYTES        ; сколько: 256 байт
    ld a,UG_TRANSPORT_BULK_VERIFIED ; передача с проверкой целостности
    call UG_WRITE
    ret
```

У `UG_WRITE` параметры отличаются от `rpc`: **HL выбирает объект на карте, DE указывает исходный массив Z80, BC задаёт количество байтов**. `UG_OFFSET` задаёт смещение внутри объекта. Сейчас оно равно нулю, поэтому запись начинается с первого байта. Смещение четырёхбайтовое; Z80 записывает его двумя двухбайтовыми командами.

`UG_TRANSPORT_BULK_VERIFIED` выбирает передачу с проверкой контрольной суммы. Библиотека сама договаривается с картой о передаче, отправляет блок и проверяет результат. При ошибке она вернёт CF=1, и наш пример перейдёт к завершению. Писать свой цикл обращения к ISA-портам для этой задачи не требуется.

После успешного `UG_WRITE` копия массива находится на карте. Звук ещё не начался: карта пока не знает, с какой частотой и в каком формате его читать.

2.5. Сообщить карте, как читать массив

Одинаковые байты можно понимать как mono или stereo, как 8-битные или 16-битные значения, читать медленно или быстро. Команда **PCM_CONFIG** сообщает, как именно следует воспроизводить наш RAW-объект. В этом описании `DW` занимает два байта, `DD` - четыре, `DB` - один.

```
pcm_request:
    DW 0                    ; сюда allocate_sound запишет sound_handle
    DD TONE_RATE            ; частота: 8000 отсчётов в секунду
    DB 1                    ; один канал: mono
    DB 8                    ; 8 бит, то есть один байт на отсчёт
    DB PCM_U8               ; значения от 0 до 255, середина = 128
    DB 0                    ; внутренняя петля не используется
    DD 0                    ; отсчёты начинаются с первого байта объекта
    DD TONE_BYTES           ; длина массива в байтах
    DD 0                    ; начало внутренней петли: не используется
    DD 0                    ; конец внутренней петли: не используется
```

Первое `DW 0` - место под номер объекта. Процедура `allocate_sound` уже записала туда полученный `sound_handle`. Остальные поля описывают выбранный звук: один канал, один байт на отсчёт, частота 8000 Гц, длина 256 байт. Внутренняя петля позволила бы повторять часть массива; здесь она выключена.

```
describe_sound:
    ld a,UG_OP_PCM_CONFIG    ; команда: описать содержимое RAW
    ld hl,pcm_request        ; номер объекта, частота, формат, длина
    ld bc,UG_OP_PCM_CONFIG_REQ_LEN ; 26 байт аргументов
    ld ix,0                  ; данных в ответе нет, только успех/ошибка
    call rpc
    ret
```

SEAL означает «загрузка закончена, проверь объект». После этой команды карта разрешает использовать объект для воспроизведения. До SEAL он ещё считается загружаемым, даже если все байты уже переданы.

```
seal_request:
    DW 0                    ; сюда allocate_sound запишет sound_handle
    DB 0                    ; не задаём ожидаемую CRC всего объекта
    DD 0                    ; это поле тогда должно быть нулём

seal_sound:
    ld a,UG_OP_SEAL          ; команда: проверить и завершить загрузку
    ld hl,seal_request
    ld bc,UG_OP_SEAL_REQ_LEN ; 7 байт аргументов
    ld ix,UG_OP_SEAL_RESP_LEN ; 4 байта контрольной суммы в ответе
    call rpc
    ret
```

CRC - контрольная сумма для проверки целостности данных. В этом примере UG_WRITE уже проверил переданный блок. Поэтому дополнительное сравнение с заранее вычисленной CRC всего объекта выключено: оба поля после номера объекта нулевые. SEAL всё равно проверяет формат и возвращает вычисленную CRC. Порядок вызовов одинаков для своих звуков: загрузить данные, описать RAW, завершить загрузку, затем воспроизводить.

2.6. Включить звук

Теперь объект готов. Команде **PLAY_PCM** задаём его номер, количество повторов, громкость и положение между левым и правым выходами:

```
play_request:
    DW 0                    ; сюда allocate_sound запишет sound_handle
    DW 0                    ; повторять весь объект до команды STOP
    DB TONE_GAIN            ; небольшая громкость, 16 из 64
    DB PAN_CENTER           ; одинаковая громкость слева и справа
```

Первое слово уже заполнено номером объекта. Второе слово равно нулю: для этой команды **ноль означает повторять весь объект до STOP**. Значение 1 означало бы один проход, то есть для нашего массива всего 0,032 секунды. Pan=128 воспроизводит mono с одинаковой громкостью слева и справа; 0 перемещает звук влево, 255 - вправо.

```
start_sound:
    ld a,UG_OP_PLAY_PCM      ; команда: воспроизвести объект
    ld hl,play_request       ; номер объекта, повторы, громкость, панорама
    ld bc,UG_OP_PLAY_PCM_REQ_LEN ; 6 байт аргументов
    ld ix,UG_OP_PLAY_PCM_RESP_LEN ; 2 байта: номер запуска
    call rpc
    ret c
    ld hl,(response)
    ld (play_id),hl           ; нужен для остановки именно этого запуска
    xor a
    ret
```

После успешного вызова слышен низкий непрерывный тон. Карта повторяет массив сама: Z80 не должен снова и снова передавать те же 256 байт.

В ответ приходит **play_id - номер запуска**. Это другой номер, чем `sound_handle`. Номер объекта выбирает хранящиеся данные; номер запуска выбирает их текущее воспроизведение. Например, объект 5 может быть запущен с номером запуска 12. Останавливать нужно запуск 12, а удалять из памяти - объект 5. Оба номера в примере сохраняются из ответов карты.

2.7. Остановить звук и вернуть память

Остановка и удаление данных - два отдельных действия. Чтобы остановить наш тон, отправляем CONTROL с номером запуска и действием STOP:

```
stop_request:
    DW 0                     ; сюда stop_sound запишет play_id
    DB 0                     ; действие 0 = STOP (остановить)

stop_sound:
    ld hl,(play_id)           ; номер запуска, который требуется остановить
    ld (stop_request),hl
    ld a,UG_OP_CONTROL       ; управление воспроизведением
    ld hl,stop_request       ; номер запуска и действие STOP
    ld bc,UG_OP_CONTROL_REQ_LEN ; 3 байта аргументов
    ld ix,0                  ; ответ без дополнительных данных
    call rpc
    ret
```

После остановки объект остаётся на карте, и его можно было бы запустить снова. В нашем примере он больше не нужен, поэтому удаляем его командой FREE:

```
free_sound:
    ld a,UG_OP_FREE          ; команда: удалить объект из памяти карты
    ld hl,sound_handle       ; адрес переменной с номером объекта
    ld bc,UG_OP_FREE_REQ_LEN ; номер занимает 2 байта
    ld ix,0                  ; ответ без дополнительных данных
    call rpc
    ret
```

Здесь `LD HL,sound_handle` передаёт **адрес переменной** с номером объекта, потому что `rpc` читает аргументы по адресу HL. При загрузке в `UG_WRITE` использовалось `LD HL,(sound_handle)`: та процедура принимала сам номер. Скобки в этих двух вызовах имеют значение.

Воспроизведение останавливает звуковая часть карты. Она может ещё не успеть применить STOP к моменту первого FREE. Тогда придёт PINNED - «данные ещё используются». В полном примере процедура `cleanup` ограниченно ждёт и повторяет FREE. Она вызывается и при обычном выходе, и при ESC, и при ошибке. После очистки UG_CLOSE завершает работу с библиотекой. Сам UG_CLOSE не заменяет остановку звука и освобождение его памяти.

2.8. Дополнительная проверка: прочитать свои байты обратно

Чтобы убедиться, что в памяти карты лежит именно наш массив, пример также вызывает `verify_sound` перед воспроизведением. Эта проверка полезна при отладке загрузчика. Для самого звучания чтение обратно не требуется.

```
verify_sound:
    ld hl,(sound_handle)     ; читаем тот же объект
    ld de,readback           ; но в другой массив памяти Z80
    ld bc,TONE_BYTES
    ld a,UG_TRANSPORT_BULK_VERIFIED
    call UG_READ             ; UG_OFFSET остался равен нулю
    ret c
    ld hl,readback
    ld de,samples
    ld b,0                   ; цикл DJNZ при B=0 выполнит 256 сравнений
    call compare             ; проверить все прочитанные байты
    ret
```

`readback` - отдельный буфер на 256 байт в памяти Z80. Подпрограмма `compare` сравнивает B байт по адресам HL и DE и при первом отличии возвращает CF=1 с кодом STATE:

```
; HL и DE = два массива, B = число байт. При первом отличии CF=1, A = STATE.
compare:
    ld a,(de)
    cp (hl)
    jr nz,.bad
    inc hl
    inc de
    djnz compare
    xor a
    ret
.bad:
    ld a,UG_ERR_STATE        ; байты не совпали
    scf
    ret
```

Ноль в B использован из-за поведения команды DJNZ: после первого уменьшения он становится 255, поэтому весь цикл выполняется 256 раз.

2.9. Вся последовательность и первые изменения

Все разобранные шаги вызываются одной процедурой. `RET C` после каждого шага означает: если он не удался, дальнейшие действия со звуком не выполнять. Выход из `play_tone` всегда ведёт к общей очистке программы.

```
play_tone:
    call allocate_sound      ; получить место в памяти карты
    ret c
    call upload_sound        ; скопировать туда массив samples
    ret c
    call describe_sound      ; задать частоту и формат массива
    ret c
    call seal_sound          ; подтвердить: загрузка закончена
    ret c
    call verify_sound        ; прочитать обратно и сравнить
    ret c
    call start_sound         ; теперь включить воспроизведение
    ret c
    call wait_tone           ; подождать 1-2 секунды или ESC
    ret
```

`wait_tone` ждёт по часам DSS, проверяет ESC и наличие ошибки звука. Через 1-2 секунды или при ESC управление возвращается к общему выходу. Ожидания ограничены, поэтому неисправность карты или часов не должна повесить программу.

Попробуйте изменить один параметр и сравнить результат:

Изменение	Что должно произойти
`TONE_RATE = 16000` вместо 8000	Тот же массив читается вдвое быстрее: тон станет 250 Гц
`TONE_GAIN = 8` вместо 16	Тон станет тише
Последнее поле play_request = 0 вместо PAN_CENTER	Mono-тон будет слышен слева
Последнее поле play_request = 255	Mono-тон будет слышен справа

Частоту и громкость изменяйте в указанных пределах API: 8000..48000 Гц и 0..64. `TONE_BYTES` должен совпадать с действительным размером массива; проверка ASSERT в исходнике обнаружит несовпадение при сборке. Для WAV или MOD сохраняется та же схема: получить номер объекта, загрузить данные, завершить загрузку, запустить и позднее освободить.

3. Обмен с картой

3.1. Запрос и ответ

Каждая команда расширенного API - это **запрос** с номером команды и аргументами и **ответ** со статусом и данными. Аргументы и данные ответа называются полезной нагрузкой (payload); её размер - от 0 до 128 байт. Многобайтовые числа представляются в формате little-endian, то есть передаются младшим байтом вперёд: число 1234h занимает два байта 34h, 12h. Выравнивания нет: поля идут подряд, как они объявлены директивами `DB`, `DW`, `DD` в примерах. Точные форматы всех команд собраны в приложении 11.6; смещения там считаются от начала полезной нагрузки.

Одновременно выполняется один запрос: следующий отправляют, получив ответ на предыдущий. Библиотека выполняет это правило сама. Статус 0 означает успех; любой другой - код ошибки из приложения 11.2, и тогда данных ответа нет. `UG_RPC` превращает ненулевой статус в CF=1 и код в A, так что проверка `RET C` после вызова охватывает и ошибки карты, и ошибки обмена.

Длину ответа проверяют всегда. Ответы фиксированного размера известны заранее (`UG_OP_…_RESP_LEN`); процедура `rpc` из главы 2 сравнивает BC с ожидаемым значением и возвращает LENGTH при расхождении. Это защищает от неверно собранного запроса и от карты другой версии.

Первые команды любой программы - INFO и CAPS. INFO сообщает, что перед нами UltraGS нужной версии (подпись UGS1, API 1.0, раскладка VLS1), что карта загрузилась без ошибок (`boot_state=0`) и каков текущий номер сеанса `session` (3.4). CAPS перечисляет доступные функции битами поля `features` и поддержанные форматы битами `formats` (11.1). Проверяйте именно тот бит, который нужен программе: так она честно откажется работать на прошивке без нужной функции, вместо того чтобы получить непонятную ошибку позже.

Так эти проверки выполняет `check_card` из главы 2. Ожидаемый ответ INFO записан в программе как подпись: первые восемь байт - `UGS1`, API 1.0, режим 8 и `boot_state=0`, следующие четыре - раскладка `VLS1`, которая в ответе лежит со смещения 36:

```
signature:
    DB "UGS1",1,0,8,0        ; magic, API 1.0, режим 8, boot_state 0
    DB "VLS1"                ; раскладка, смещение 36 в ответе INFO
```

После первого `compare` регистр DE уже указывает на `VLS1`, поэтому второй вызов сравнивает раскладку без новой загрузки DE:

```
check_card:
    ld a,UG_OP_INFO
    ld bc,0
    ld ix,UG_OP_INFO_RESP_LEN
    call rpc
    ret c
    ld hl,response
    ld de,signature
    ld b,8
    call compare
    ret c
    ld hl,response+36
    ld b,4
    call compare
    ret c
```

Затем процедура проверяет бит 2 поля `features` (передача данных блоками, которой пользуется `UG_WRITE`) и бит 0 поля `formats` (RAW 8 бит); несовпадение даёт STATE. Последний шаг - сброс карты `reset_card`, разобранный в 3.5:

```
    ld a,UG_OP_CAPS
    ld bc,0
    ld ix,UG_OP_CAPS_RESP_LEN
    call rpc
    ret c
    ld a,(response)
    bit 2,a                 ; бит 2: передача данных блоками
    jr z,.unsupported
    ld a,(response+4)
    bit 0,a                 ; карта понимает формат RAW8
    jr z,.unsupported
    jp reset_card           ; начать с чистой карты
.unsupported:
    ld a,UG_ERR_STATE
    scf
    ret
```

Контрольная сумма команд включается вызовом `UG_FRAME_CRC` с A=1: после этого библиотека добавляет CRC32 к каждому запросу, а карта - к каждому ответу, и повреждённый обмен обнаруживается кодом CRC. Стоит это несколько десятков микросекунд на команду; примеры руководства включают проверку сразу после `UG_OPEN`.

3.2. Передача данных блоками

Для звуковых данных есть отдельный механизм - **блочная передача**. Один блок содержит от 1 до 1024 байт. Карта разрешает передачу, принимает ровно столько байт, сколько было объявлено, и подтверждает блок. Библиотечные `UG_WRITE` и `UG_READ` скрывают эти шаги: программа указывает объект, буфер Z80, длину и смещение `UG_OFFSET` внутри объекта.

Загрузка идёт последовательно: каждый следующий блок начинается там, где закончился предыдущий, поэтому после удачного `UG_WRITE` программа прибавляет длину блока к `UG_OFFSET`. Так загружается файл любого размера - глава 6 показывает этот цикл. Читать объект можно с любого смещения в пределах уже загруженного, в том числе во время его воспроизведения; записывать в готовый (READY) объект нельзя.

Способ передачи выбирает регистр A. `UG_TRANSPORT_BULK_VERIFIED` (80h) добавляет к блоку контрольную сумму и даёт уверенность, что данные дошли без искажений; `UG_TRANSPORT_BULK_FAST` (00h) экономит время на длинных загрузках, когда целостность проверят иначе, например SEAL с известной CRC всего файла. Варианты 01h и 81h передают данные через порт GS DATA и нужны только программам, ограниченным протоколом GS.

Пока блок передаётся, карта принимает лишь служебные команды: INFO, MEM_INFO, STATS, SNAPSHOT, страницы состояния, OBJECT_INFO текущего объекта и завершение или отмену самой передачи. Остальные вернут BUSY. При работе через библиотеку это не создаёт трудностей: `UG_WRITE` возвращается только после завершения блока.

3.3. Ошибки и потеря ответа

Коды ошибок перечислены в приложении 11.2 вместе с рекомендуемым действием. На практике важны несколько ситуаций.

- **BUSY** - карта занята другой операцией. Ожидание с повтором допустимо, но число попыток или время должны быть ограничены, иначе программа зависнет при неисправности.
- **PINNED** - объект ещё используется запуском или передачей. Остановите свои запуски и повторите FREE через короткое время, как делает `cleanup` в примерах.
- **NO_VOICE** - для нового звука нет свободных голосов. Эффект можно пропустить или запустить позже; останавливать чужую музыку ради него нельзя.
- **STATE** и **HANDLE** - команда не подходит к текущему состоянию объекта или номер неизвестен карте. Часто это признак того, что номер устарел после смены сеанса (3.4).
- **TIMEOUT** - карта не ответила за отведённое библиотекой время.

Отдельный случай - команда выполнена, но ответ потерян: например, PLAY уже включил звук, а программа не получила play_id. Повторять такую команду вслепую нельзя - второй PLAY запустит звук ещё раз. Правильный порядок: сбросить обмен (`UG_XCTRL` с `UG_XC_TRANSPORT_RESET`), заново прочитать INFO и убедиться, что сеанс прежний, затем узнать фактическое состояние через SNAPSHOT, OBJECT_INFO или STREAM_STATE и продолжить с него. Сброс обмена отменяет неполную передачу, но не откатывает уже выполненные команды.

3.4. Сеанс

`session` - восьмибайтовый идентификатор текущего состояния памяти карты. Карта меняет его при сбросе хранилища и при перезапуске. Программа его не выбирает и в команды не передаёт; он нужен, чтобы вовремя заметить, что сохранённые номера объектов и запусков обесценились.

Представьте: программа загрузила звук и получила handle 5. Карту сбросили, звук исчез, и позже номер 5 достался другому объекту. Продолжая пользоваться сохранённой пятёркой, программа обратилась бы к чужим данным. Сравнение сеанса защищает от этого. Сохраните `session` в начале работы: `reset_card` (3.5) берёт его из ответа RESET_STORE, а программа, которая карту не сбрасывает, - из поля `session` ответа INFO (смещение 12). Перед восстановлением после ошибки прочитайте INFO снова и сравните восемь байт с сохранёнными; если они изменились, старые номера нужно забыть и ничего по ним не удалять. Так делает общий выход `cleanup` в главе 2:

```
ld a,UG_OP_INFO          ; прочитать INFO заново
ld bc,0
ld ix,UG_OP_INFO_RESP_LEN
call rpc
ret c
ld hl,response+12        ; поле session в ответе INFO
ld de,saved_session      ; сеанс, сохранённый при сбросе карты
ld b,8
call compare
jr c,.changed            ; сеанс сменился: старые номера забыть
```

`compare` возвращает CF=1 при первом отличии; метка `.changed` завершает очистку с кодом STATE, не трогая объекты нового сеанса.

3.5. Исходное состояние карты

Карта не следит за тем, какая программа что загрузила. После предыдущей программы на ней могут остаться объекты, играющая музыка, эффекты, открытый поток или брошенная на середине передача - особенно если та программа завершилась аварийно. Поэтому программа, которой нужна вся карта, начинает с команды **RESET_STORE** (06h). Запрос состоит из ключа и уровня. Ключ 55475253h - это буквы «UGRS» (UltraGS Reset Store), прочитанные как число u32; он защищает от случайного сброса: с другим значением карта отвечает BAD_ARGUMENT. Уровень 0 - тёплый сброс: карта останавливает всё воспроизведение, удаляет все объекты и отменяет передачи, а настройки GS (команды 6Ah и 6Bh) сохраняет. Уровень 1 - холодный: сбрасываются и эти настройки, после чего выполняется короткая проверка памяти; он нужен диагностическим утилитам, а не обычной программе. Ответ - новый `session` (3.4); все прежние номера объектов и запусков недействительны.

Так это делает `reset_card` из главы 2. Перед сбросом она подаёт `UG_XCTRL` с `UG_XC_TRANSPORT_RESET`: если предыдущая программа оборвалась посреди передачи блока, карта сама закроет эту передачу только через пять секунд без данных и до тех пор отвечала бы на RESET_STORE кодом BUSY, а сброс обмена снимает её сразу:

```
; Привести карту в исходное состояние: остановить всё, что играет, удалить
; все объекты и отменить незавершённые передачи. Ответ - новый номер сеанса.
reset_card:
    ld a,UG_XC_TRANSPORT_RESET ; снять передачу, брошенную прошлой программой
    call UG_XCTRL
    ret c
    ld a,UG_OP_RESET_STORE
    ld hl,reset_request
    ld bc,UG_RESET_STORE_REQ_LEN
    ld ix,UG_OP_RESET_STORE_RESP_LEN
    call rpc
    ret c
    ld hl,response           ; ответ целиком - новый session
    ld de,saved_session      ; его копия в памяти Z80
    ld bc,8                  ; идентификатор занимает восемь байт
    ldir
    xor a
    ret

reset_request:
    DD UG_RESET_STORE_KEY    ; ключ 55475253h, буквы «UGRS»
    DB 0                     ; уровень 0: тёплый сброс (настройки GS остаются)
```

`reset_request` - пять байт запроса, объявленные директивами `DD` и `DB`; `UG_RESET_STORE_KEY`, `UG_RESET_STORE_REQ_LEN` и `UG_OP_RESET_STORE_RESP_LEN` определены в `ug_wire.inc`. Ещё один случай BUSY - незакрытая загрузка файла командами GS (между 30h и D2h): её закрывает команда D2h (глава 9). Программа, работающая только командами GS, выполняет тот же тёплый сброс командой F3h.

Карту сбрасывают в начале работы, а не при выходе: при выходе программа освобождает только свои объекты и останавливает только свои запуски (4.4). Не сбрасывает карту программа, которая рассчитывает на объект, загруженный заранее другой программой: вместо RESET_STORE она читает MEM_INFO и OBJECT_ENUM (глава 4) и работает с тем, что уже есть.

4. Объекты и память

4.1. Жизненный цикл объекта

Любой звук проходит одни и те же состояния. ALLOC создаёт объект нужного вида и размера и возвращает handle; объект находится в состоянии UPLOADING. `UG_WRITE` переносит данные. Для RAW дополнительно нужен PCM_CONFIG. SEAL проверяет содержимое и переводит объект в READY; с этого момента данные неизменяемы, и объект можно воспроизводить сколько угодно раз. FREE удаляет объект, когда ни один запуск и ни одна передача его не используют.

Вид (kind)	Константа	Что хранится	Что нужно до SEAL
0	`UG_KIND_BLOB`	Произвольные байты, не воспроизводятся	Ничего
1	`UG_KIND_RAW_PCM`	Отсчёты без заголовка	PCM_CONFIG
2	`UG_KIND_WAV_PCM`	Файл RIFF/WAVE целиком	Ничего: заголовок разбирает карта
3	`UG_KIND_MOD`	Файл MOD целиком	Ничего
5	`UG_KIND_S3M`	Файл S3M целиком	Ничего

Размер в ALLOC - точный размер данных в байтах; карта округляет резерв вверх до 16 байт и сообщает его в ответе. Память выделяется одним непрерывным участком, поэтому для большого объекта важен не общий остаток, а наибольший свободный участок `largest` из MEM_INFO. Одновременно в сеансе живут не более 128 объектов. Номера объектов не переиспользуются до сброса карты, поэтому программа, которая много раз создаёт и удаляет объекты, в конце концов получит NO_HANDLES; счётчик номеров обнуляет только RESET_STORE (3.5).

SEAL с флагом 0 просто проверяет формат. С флагом 1 карта дополнительно сравнивает CRC32 всего файла с переданным значением - удобно, когда файл загружали быстрым способом без проверки блоков. SEAL не повторяют для READY объекта.

Pin - признак того, что объект занят. Его ставят воспроизведение, загрузка и чтение. FREE занятого объекта возвращает PINNED. Остановка запуска снимает pin не мгновенно, а на ближайшей границе обработки звука, поэтому после STOP первая попытка FREE может не пройти; повтор через несколько миллисекунд обычно успешен.

4.2. Что сообщить карте о данных

RAW. PCM_CONFIG описывает массив отсчётов: частота 8000..48000 Гц, каналы 1 или 2, разрядность 8 или 16, формат U8, S8 или S16LE (для 16 бит - только S16LE), начало и длина данных в байтах, при желании внутренняя петля. Длина должна быть кратна размеру кадра: кадр - это отсчёты всех каналов в один момент времени, то есть 1 байт для U8 mono и 4 байта для S16 stereo. Границы петли задаются в кадрах, а не в байтах, и действуют при воспроизведении с одним проходом (repeats=1).

WAV. Карта принимает WAV с одним блоком `fmt` и одним блоком `data`, mono или stereo, с частотой в тех же пределах, в двух вариантах: обычный PCM (8 или 16 бит) и сжатый IMA ADPCM (кодек 0x11, 4 бита на отсчёт, блок до 8192 байт). ADPCM-файл занимает на карте вчетверо меньше места, чем 16-битный PCM, и во столько же раз быстрее грузится; карта распаковывает его при воспроизведении, а блок `fact`, если он есть, задаёт точную длину. Такой файл готовит, например, `ffmpeg -i in.wav -c:a adpcm_ima_wav out.wav`. Посторонние блоки и выравнивание допускаются. Другие сжатые, float- и расширенные форматы,

RIFX и RF64 отвергаются с кодом FORMAT. PCM_CONFIG для WAV не нужен и запрещён.

MOD. Поддержаны модули с 1..32 каналами: `M.K.`, `M!K!`, `FLT4`, `8CHN`, `FLT8`, `1CHN`..`9CHN`, `10CH`..`32CH`. Файл загружается целиком, как есть; SEAL разбирает заголовок и проверяет паттерны. Четырёхканальный модуль занимает четыре голоса, восьмиканальный - восемь, и так далее.

S3M. Поддержан профиль с PCM-инструментами (без FM-инструментов и упакованных образцов), до 32 каналов. Канал с mono-инструментами занимает один голос, канал, использующий stereo-инструмент, - два. SEAL строит для S3M дополнительный индекс в памяти карты, поэтому объекту нужно немного больше места, чем размер файла; MEM_INFO учитывает оба участка, FREE освобождает оба. Управляют S3M теми же командами, что и MOD.

4.3. Что можно узнать об объекте

OBJECT_INFO возвращает вид, состояние (0 загрузка, 1 готов, 2 испорчен), число pin, размер, сколько байт уже загружено, CRC и разобранные параметры звука: частоту, число каналов, разрядность и формат отсчётов (у IMA ADPCM - 4 бита и формат 3), длину в кадрах, у модулей - число каналов трекера. OBJECT_ENUM перечисляет номера всех объектов по возрастанию: первый вызов с 0, следующий - с полученного номера, ноль завершает обход. Перечисление показывает все объекты карты, а не только созданные вашей программой: о принадлежности оно ничего не говорит.

MEM_INFO сообщает физический объём памяти, размер пула, свободный объём, наибольший непрерывный участок и число объектов. Для оценки «поместится ли файл» нужен `largest`.

4.4. Что освобождать при выходе

Программа запоминает номера всех своих объектов и запусков. При выходе - обычном, по ESC или по ошибке - она:

1. завершает или отменяет свою передачу блока;
2. останавливает свои запуски: CONTROL со своим play_id, STOP_FX со своими instance_id;
3. дожидается снятия pin и удаляет свои объекты командой FREE, повторяя её при PINNED ограниченное число раз.

Чужие звуки при выходе не трогают. STOP_FX с нулевым номером останавливает все эффекты карты, RESET_STORE стирает всё хранилище - это действия для начала работы (3.5), а не для выхода. Объект, который должен пережить программу (например, загруженный утилитой UGLOAD для следующих программ), просто не удаляют.

5. Воспроизведение

5.1. Основной звук

PLAY_PCM запускает готовый RAW или WAV как основной звук. Поле `repeats` задаёт число проходов: 1 - один раз, 2..65535 - столько раз, 0 - повторять до STOP. `gain` - громкость 0..64, `pan` - положение 0..255 (0 слева, 128 по центру, 255 справа). В ответ приходит play_id.

PLAY_MOD запускает MOD или S3M с начала и тоже возвращает play_id. Флаги: `UG_MOD_LOOP` (1) повторяет песню после конца; `UG_MOD_IGNORE_F00` (2) и `UG_MOD_PAUSE_F00` (4) задают, что делать с командой F00 внутри модуля - играть дальше или остановиться с сохранением позиции. Два последних флага несовместимы друг с другом и требуют бита 12 в CAPS; без них действует настройка карты (команда GS 6Ah). Для S3M допустим только LOOP.

Новый PLAY основного звука заменяет предыдущий, но только после полной проверки нового запуска: если новому не хватает голосов или объект не готов, прежняя музыка продолжает играть, а команда возвращает ошибку.

CONTROL принимает play_id и действие: 0 STOP, 1 PAUSE, 2 RESUME. Пауза сохраняет голоса и позицию; продолжить можно только из паузы, а не после конца записи. Ноль вместо play_id означает «текущий основной звук», но для очистки лучше передавать свой точный номер: так программа не остановит музыку, запущенную другой программой после неё.

Трекером управляют ещё три команды. **SEEK_MOD** переходит на позицию `order` и строку 0..63, сбрасывая эффекты каналов и темп на значения по умолчанию. **SET_TEMPO** меняет скорость (1..31 для MOD, 1..255 для S3M) и BPM (32..255); команды темпа внутри самой песни позже могут изменить их снова. **SET_PAN** задаёт панораму логическому каналу трекера 0..31, пока следующий PLAY, SEEK или команда панорамы в модуле её не заменят.

SET_MIX устанавливает четыре общих уровня 0..64: мастер-громкость музыки и эффектов и дополнительные множители для них. Это общая настройка карты, она действует на все запуски, включая запущенные командами GS, и не влияет на OPL2.

5.2. Эффекты

PLAY_FX запускает готовый RAW или WAV независимо от основного звука и возвращает instance_id вместе с числом занятых голосов. Поля запроса: handle, предпочтительный голос (`UG_VOICE_AUTO` = FFFFh, чтобы карта выбрала сама), приоритет 1..254, громкость, панорама и флаги. Флаг 1 разрешает вытеснить эффект с меньшим приоритетом, когда свободных голосов нет; флаг 2 повторяет весь объект до остановки. Музыку эффект вытеснить не может. Если голосов не хватает, stereo-эффект не стартует даже наполовину: ответ NO_VOICE означает, что не изменилось ничего.

STOP_FX останавливает один эффект по его instance_id; объект остаётся на карте. **CONTROL_FX** приостанавливает и продолжает эффект тем же набором действий, что CONTROL. Нулевой instance_id в STOP_FX останавливает все эффекты карты; в обычной программе он не нужен.

Эффект продолжает звучать при паузе и остановке музыки: это разные запуски. Чтобы при выходе ничего не осталось играть, программа останавливает каждый свой эффект отдельно.

5.3. Громкость, панорама и запас

Все громкости задаются в диапазоне 0..64, где 64 - без ослабления. Для mono-звука панорама распределяет сигнал между левым и правым выходами; для stereo она работает как баланс, и в центре оба канала звучат без ослабления.

Сложение многих источников может превысить диапазон 16-битного выхода - это слышно как искажение. Карта считает такие случаи в счётчике `clip_samples` (STATS). Чтобы оставить запас, модули с 5..8

каналами ослабляются вдвое, с 9..16 - вчетверо, с 17..32 - в восемь раз до панорамирования; четырёхканальные модули, PCM и эффекты играют без ослабления. Если эффекты поверх громкой музыки искажаются, уменьшите их gain или общий уровень через SET_MIX.

5.4. Голоса

Пул из 32 голосов общий для музыки и эффектов. Восьмиканальный MOD занимает восемь; оставшихся 24 хватит, например, на 24 mono- или 12 stereo-эффектов. MOD с 32 каналами или S3M с 16 stereo-каналами занимает всё, и эффекты получат NO_VOICE. Пауза голоса не освобождает: при продолжении звук должен начаться ровно с того места, где остановился. Сколько голосов занято сейчас, показывает поле `voice_count` снимка состояния (глава 8).

6. Пример: музыка и эффект поверх неё

Задача: включить фоновую музыку из MOD-файла, через секунду добавить поверх неё короткий эффект, ещё через секунду убрать эффект, а в конце всё остановить и освободить. Полный текст - modfx.asm; MOD-файл `GUIDE.MOD` создаётся при сборке примеров и включён в программу директивой `INCBIN`. Открытие карты, проверки INFO/CAPS, сброс карты `reset_card` и процедуры `rpc` и `compare` те же, что в главе 2, и здесь не повторяются. `wait_seconds` ждёт A секунд по часам DSS так же, как `wait_tone` в главе 2: ESC прерывает ожидание с кодом CANCELLED, а ошибка звуковой части, замеченная через SNAPSHOT, - с её кодом. Переменные `music_handle`, `fx_handle`, `play_id` и `fx_id` в начале программы обнуляются: ноль означает «этого ресурса нет».

```
BLOCK_BYTES EQU 1024         ; наибольший блок одной передачи UG_WRITE
TONE_RATE   EQU 8000         ; частота гудка-эффекта
TONE_BYTES  EQU 256          ; длина его массива
FX_GAIN     EQU 32           ; громкость эффекта: 0 = тишина, 64 = максимум
FX_REPEAT   EQU 2            ; флаг PLAY_FX: повторять весь объект
```

Порядок действий читается прямо из главной процедуры. Каждый шаг - отдельная подпрограмма с `RET C` после неё:

```
play_lesson:
    call load_music          ; ALLOC, блоки UG_WRITE, SEAL
    ret c
    call load_effect         ; то же для гудка плюс PCM_CONFIG
    ret c
    call start_music         ; PLAY_MOD с повтором песни
    ret c
    ld a,1
    call wait_seconds        ; секунда одной музыки
    ret c
    call start_effect        ; PLAY_FX поверх музыки
    ret c
    ld a,1
    call wait_seconds        ; секунда музыки вместе с гудком
    ret c
    call stop_effect         ; STOP_FX: музыка продолжается
    ret c
    ld a,1
    jp wait_seconds          ; ещё секунда одной музыки
```

6.1. Загрузка файла блоками

Файл `GUIDE.MOD` включён в программу целиком, и его размер известен при сборке:

```
music_data:
    INCBIN "GUIDE.MOD"       ; файл MOD включается в программу целиком
MUSIC_BYTES EQU $-music_data ; его размер в байтах
```

Файл занимает 2172 байта, а один вызов `UG_WRITE` передаёт не больше 1024 байт (`BLOCK_BYTES`), поэтому загрузка идёт в три приёма: 1024, 1024 и 124 байта. `load_music` выделяет объект вида MOD на весь размер файла, а затем в цикле передаёт по блоку, каждый раз сдвигая `UG_OFFSET` и адрес источника; переменная `remaining` хранит число ещё не переданных байтов. После последнего блока SEAL проверяет заголовок и паттерны модуля. PCM_CONFIG для MOD не нужен: карта сама знает его формат.

```
load_music:
    ld a,UG_OP_ALLOC
    ld hl,music_alloc        ; kind=MOD, размер файла
    ld bc,UG_OP_ALLOC_REQ_LEN
    ld ix,UG_OP_ALLOC_RESP_LEN
    call rpc
    ret c
    ld hl,(response)
    ld (music_handle),hl
    ld (music_seal),hl
    ld (music_play),hl
    ld hl,0
    ld (UG_OFFSET),hl        ; первый блок пишется с начала объекта
```

```
    ld (UG_OFFSET+2),hl
    ld hl,MUSIC_BYTES
    ld (remaining),hl
    ld de,music_data
.block:
    ld bc,(remaining)
    ld hl,BLOCK_BYTES
    or a
    sbc hl,bc               ; осталось больше одного блока?
    jr nc,.size_ok
    ld bc,BLOCK_BYTES       ; да: передаём ровно 1024 байта
.size_ok:
    push bc
    push de
    ld hl,(music_handle)
    ld a,UG_TRANSPORT_BULK_VERIFIED
    call UG_WRITE           ; HL=объект, DE=источник, BC=длина
    pop de
    pop bc
    ret c
    ld hl,(UG_OFFSET)
    add hl,bc               ; следующий блок продолжает предыдущий
    ld (UG_OFFSET),hl
    ex de,hl
    add hl,bc               ; источник тоже сдвигается
    ex de,hl
    ld hl,(remaining)
    or a
    sbc hl,bc
    ld (remaining),hl
    ld a,h
    or l
    jr nz,.block
    ld a,UG_OP_SEAL         ; все байты на карте: проверить формат MOD
    ld hl,music_seal
    ld bc,UG_OP_SEAL_REQ_LEN
    ld ix,UG_OP_SEAL_RESP_LEN
    jp rpc
```

Вычитание `SBC HL,BC` сравнивает остаток с размером блока: если остаток больше, перенос установлен и блок ограничивается 1024 байтами. `UG_WRITE` может изменить BC и DE, поэтому они сохраняются на стеке вокруг вызова. Так же загружается файл, прочитанный с диска по частям: читать очередную порцию можно между вызовами `UG_WRITE`, пока окно ISA закрыто.

Эффект в этом примере - тот же 256-байтовый гудок, что в главе 2, и `load_effect` повторяет шаги 2.3-2.5: ALLOC, UG_WRITE со смещением 0, PCM_CONFIG, SEAL. Обратите внимание: перед записью во второй объект `UG_OFFSET` снова обнуляется - смещение относится к объекту, а не к карте.

6.2. Запуск музыки и эффекта

```
start_music:
    ld a,UG_OP_PLAY_MOD
    ld hl,music_play        ; номер объекта и флаги
    ld bc,UG_OP_PLAY_MOD_REQ_LEN
    ld ix,UG_OP_PLAY_MOD_RESP_LEN
    call rpc
    ret c
    ld hl,(response)
    ld (play_id),hl         ; номер запуска музыки
    xor a
    ret
```

PLAY_MOD получает номер объекта и флаги; `UG_MOD_LOOP` заставляет песню повторяться, пока её не остановят. Через секунду поверх музыки включается эффект:

```
start_effect:
    ld a,UG_OP_PLAY_FX
    ld hl,fx_play           ; объект, выбор голоса, приоритет, громкость
    ld bc,UG_OP_PLAY_FX_REQ_LEN
    ld ix,UG_OP_PLAY_FX_RESP_LEN
    call rpc
    ret c
    ld hl,(response)        ; первые 2 байта: номер запуска эффекта
    ld (fx_id),hl           ; следующие 2: сколько голосов он занял
    xor a
    ret

stop_effect:
    ld hl,(fx_id)
    ld (fx_stop),hl
    ld a,UG_OP_STOP_FX
    ld hl,fx_stop
    ld bc,UG_OP_STOP_FX_REQ_LEN
    ld ix,0
    call rpc
    ret c
    ld hl,0
    ld (fx_id),hl           ; больше останавливать нечего
    ret
```

PLAY_FX не трогает музыку: она продолжает играть всеми голосами, а эффект получает свой голос. Флаг FX_REPEAT повторяет короткий массив, поэтому гудок слышен ровно столько, сколько пройдёт до STOP_FX. После остановки эффекта музыка звучит одна ещё секунду. Запросы команд объявлены в одном месте; номера объектов в них записывают процедуры загрузки:

```
music_alloc:
    DB UG_KIND_MOD          ; карта сама разберёт заголовок MOD
    DD MUSIC_BYTES          ; размер файла в байтах
music_seal:
    DW 0                    ; номер объекта запишет load_music
    DB 0                    ; CRC всего файла не задаём
    DD 0
music_play:
    DW 0                    ; номер объекта
    DB UG_MOD_LOOP          ; повторять песню до STOP
music_stop:
    DW 0                    ; номер запуска
    DB 0                    ; действие STOP
fx_alloc:
    DB UG_KIND_RAW_PCM
    DD TONE_BYTES
fx_pcm:
    DW 0                    ; номер объекта
    DD TONE_RATE            ; 8000 отсчётов в секунду
    DB 1,8,0,0              ; mono, 8 бит, U8, без внутренней петли
    DD 0                    ; данные с начала объекта
    DD TONE_BYTES
    DD 0,0
fx_seal:
    DW 0
    DB 0
    DD 0
fx_play:
    DW 0                    ; номер объекта
    DW UG_VOICE_AUTO        ; голос выберет карта
    DB 128                  ; приоритет среди эффектов
    DB FX_GAIN
    DB 128                  ; панорама: по центру
    DB FX_REPEAT            ; повторять, пока не остановим
fx_stop:
    DW 0                    ; номер запуска эффекта
free_request:
    DW 0
```

6.3. Очистка

Общий выход вызывается всегда: после успешного ожидания, по ESC и при любой ошибке. Он останавливает ровно те запуски и удаляет ровно те объекты, номера которых программа успела сохранить; нулевой номер означает «этого шага не было». FREE повторяется, пока звуковая часть не применит STOP.

`stop_music` устроена как `stop_sound` в 2.7: CONTROL с действием STOP и своим `play_id`, после чего `play_id` обнуляется. `snapshot` запрашивает SNAPSHOT через `rpc`; в `free_object` он служит короткой паузой перед повтором FREE и заодно проверяет, что карта отвечает:

```
snapshot:
    ld a,UG_OP_SNAPSHOT      ; снимок состояния: 64 байта в response
    ld bc,0
    ld ix,UG_OP_SNAPSHOT_RESP_LEN
    jp rpc

; Общий выход: при успехе, ESC и любой ошибке. Останавливает только свои
; запуски и удаляет только свои объекты, повторяя FREE, пока данные заняты.
cleanup:
    ld a,UG_XC_TRANSPORT_RESET
    call UG_XCTRL            ; закрыть недокончennый обмен, если он был
    ret c
    ld hl,(fx_id)
    ld a,h
    or l
    call nz,stop_effect
    ret c
    ld hl,(play_id)
    ld a,h
    or l
    call nz,stop_music
    ret c
    ld hl,fx_handle
    call free_object
    ret c
    ld hl,music_handle
    jp free_object

; HL = адрес переменной с номером объекта; 0 означает "удалять нечего".
free_object:
    ld e,(hl)
    inc hl
    ld d,(hl)
    ld a,d
    or e
    ret z
    ld (free_request),de
    ld a,64
    ld (retries),a
.retry:
    call snapshot            ; пауза: звуковая часть применяет STOP
    ret c
    ld a,UG_OP_FREE
    ld hl,free_request
    ld bc,UG_OP_FREE_REQ_LEN
    ld ix,0
    call rpc
    jr nc,.freed
    cp UG_ERR_PINNED         ; данные ещё звучат: повторить позже
    jr z,.again
    cp UG_ERR_BUSY
    ret nz
.again:
    ld hl,retries
    dec (hl)
    jr nz,.retry
    ld a,UG_ERR_TIMEOUT
    scf
    ret
.freed:
    xor a
    ret
```

Запустите `MODFX0` (или `MODFX1` для второго слота). Ожидается четыре такта музыки, гудок поверх второй секунды и сообщение `MODFX: music looped, effect added and stopped, freed`. ESC в любой момент ожидания завершает программу с кодом 15h (CANCELLED) после той же очистки.

7. Поток с диска

7.1. Когда нужен поток

Объект целиком хранится на карте. Для длинной записи это неудобно: файл в несколько мегабайт долго грузится и занимает память. **Поток** решает задачу иначе: карта держит в памяти кольцевой буфер, программа подливает в него данные порциями, а карта воспроизводит их по мере поступления. Так можно проиграть файл, который не хочется хранить целиком, или звук, который программа создаёт на ходу. Для коротких эффектов поток не нужен - проще загрузить объект.

7.2. Команды потока

STREAM_CREATE создаёт кольцо: в запросе формат отсчётов (как первые восемь байт PCM_DESC) и три размера в байтах - сам буфер, `prefill` и `low`. Воспроизведение начнётся, когда в кольце накопится `prefill` байт; когда запас опустится ниже `low`, карта поднимет флаг «нужно пополнение». Размеры кратны кадру. Поток на карте может быть только один, и ответ содержит его handle.

STREAM_START занимает основной звук, как PLAY_PCM, и возвращает play_id. STREAM_FEED передаёт очередную порцию: номер пакета (первый - 1), длину 1..1024 байт, кратную кадру, способ передачи и, при VERIFIED, контрольную сумму данных. В отличие от обычной команды, FEED сам содержит передачу блока: после запроса библиотека дожидается разрешения, передаёт данные и только потом читает ответ с номером следующего пакета. Отдельный WRITE_END не нужен. STREAM_EOF сообщает, что данных больше не будет; остаток доигрывается. STREAM_STATE показывает состояние и счётчики, STREAM_CLOSE освобождает кольцо. После EOF объект потока сохраняется до CLOSE, чтобы программа могла дождаться состояния «воспроизведено» и прочитать итоговые счётчики.

Состояния потока: 0 создан, 1 играет, 2 пауза, 3 конец, 4 ошибка, 5 буферизация. Буферизация наступает, когда данные закончились раньше времени: позиция сохраняется, счётчик `underruns` увеличивается на единицу, а после нового накопления `prefill` звук продолжается. Именно по этому счётчику программа узнаёт, успевала ли она подавать данные.

Если ответ на FEED потерян, пакет нельзя отправлять повторно вслепую: он мог дойти, и тогда данные задвоятся. Сброс обмена, затем STREAM_STATE: поле `next_sequence` подскажет, какой пакет карта ждёт.

7.3. Пример: UGPLAY STREAM

Поток показывает утилита UGPLAY: `UGPLAY STREAM файл.WAV` читает WAV с диска и подаёт в кольцо содержимое блока `data`. Ключи `/BUFFER:`, `/PREFILL:` и `/LOW:` задают три размера в КиБ. Для U8 mono 8000 Гц разумны `/BUFFER:16 /PREFILL:8 /LOW:4`: две секунды запаса, секунда до старта и полсекунды до опустошения при сигнале пополнения. Файл `GUIDE.WAV` из комплекта примеров подходит для проверки.

Порядок действий UGPLAY одинаков для любой программы с потоком:

1. STREAM_CREATE с форматом из заголовка WAV и тремя размерами в байтах; ответ - handle потока.
2. STREAM_START с этим handle, громкостью и панорамой; ответ - play_id.
3. Цикл до конца файла: прочитать порцию при закрытом окне ISA; выбрать длину не больше 1024 байт и не больше свободного места из STREAM_STATE, кратную кадру; отправить STREAM_FEED с номером пакета, длиной и способом передачи; дождаться бита WRITE_GRANT в X_STATUS; передать байты через порт BULK_DATA; прочитать ответ с номером следующего пакета.
4. STREAM_EOF, ожидание состояния 3 по STREAM_STATE, STREAM_CLOSE.

Форматы команд приведены в 11.6. Шаг 3 отличается от обычной команды тем, что данные передаются между запросом и ответом; в библиотеке эту передачу выполняет `UG_RPC`, собранный с определением `APP_PLAY`, как в UGPLAY. Готовая реализация цикла - ugstream.asm: разбор заголовка RIFF, выбор длины пакета, VERIFIED с CRC данных и восстановление после потерянного ответа FEED по `next_sequence`. Программа, которой нужен поток, берёт этот исходник за основу.

7.4. Ограничение

Поток 44100 Гц S16 stereo требует 176 400 байт в секунду без учёта служебного обмена. На Sprinter с чтением с диска такая скорость пока не достигнута: слышны паузы, счётчик `underruns` растёт, хотя сам обмен завершается без ошибок. Больший буфер сглаживает паузы, но не исправляет недостаточную среднюю скорость. Для mono 8 бит с частотой 8000..22050 Гц запаса достаточно. Проверяйте `underruns` до и после воспроизведения: успешные ответы FEED и достижение EOF сами по себе не доказывают, что звук шёл без пауз.

8. Наблюдение за картой

8.1. Снимок состояния

SNAPSHOT возвращает 64 байта согласованной сводки: номер снимка, сеанс, счётчик выданных кадров, номер и состояние основного запуска (0 остановлен, 1 играет, 2 пауза, 3 конец, 4 ошибка), вид источника (1 PCM, 2 трекер, 4 поток), handle объекта, позицию трекера (order, row, tick, speed, BPM), пиковые уровни левого и правого каналов, число underrun, код последней ошибки звука, количество занятых голосов и эффектов и сведения о текущей передаче. Снимок не останавливает звук и допустим в любой момент; примеры этого руководства читают его в цикле ожидания, чтобы вовремя заметить ошибку звуковой части (поле `last_audio_error`).

8.2. Голоса и эффекты постранично

Записи отдельных голосов и эффектов не помещаются в один ответ, поэтому читаются страницами. Порядок такой: получить SNAPSHOT, скопировать из него сеанс и номер снимка, затем вызывать VOICE_STATUS или FX_STATUS, передавая этот ключ, индекс первой записи и число записей (до 5 голосов или 6 эффектов за страницу). Ответ начинается заголовком с общим числом записей; индекс увеличивают на возвращённое количество, пока он не сравняется с общим. Ключ гарантирует, что все страницы описывают один и тот же момент: если карта успела сделать новый снимок, запрос вернёт STATE, и обход начинают заново с нового снимка.

VOICE_STATUS перечисляет все 32 голоса, включая свободные: для каждого - номер запуска, объект, логический канал трекера, позицию в данных, пик, период, ноту, громкость, панораму и флаги (активен, принадлежит эффекту, на паузе). FX_STATUS перечисляет только активные эффекты; stereo-эффект - одна запись с `channels=2`.

8.3. Счётчики

STATS возвращает 16 счётчиков: обращения к портам, переданные байты и блоки, ошибки CRC и протокола, тайм-ауты передач, underrun, ошибки памяти, худшее время обработки одного кванта звука и одной команды в микросекундах, пиковое заполнение очередей и число ограниченных отсчётов `clip_samples`. Счётчики общие для всей карты и растут с момента включения, поэтому программа сравнивает значения до и после своего сценария. CLEAR_STATS обнуляет их для всех; в обычной программе он не нужен.

Служебные действия `UG_XCTRL` принимают одно из четырёх значений: `UG_XC_TRANSPORT_RESET` сбрасывает обмен и отменяет неполную передачу, `UG_XC_TRANSFER_CANCEL` отменяет только передачу, `UG_XC_CLEAR_ERRORS` очищает регистры ошибок, `UG_XC_SNAP_LATCH` фиксирует снимок для чтения через порт SNAP_DATA без команды.

9. Команды General Sound

Эта глава нужна, если вы поддерживаете программу, написанную для General Sound, или хотите, чтобы новая программа работала и с настоящей GS. Для новых программ только под UltraGS удобнее расширенный API: он возвращает коды ошибок и длины ответов, тогда как GS сообщает об ошибках отдельным регистром.

9.1. Как устроен обмен GS

GS управляется через два регистра. **DATA** передаёт байты данных в обе стороны, **COMMAND** принимает код команды, а при чтении отдаёт байт состояния. В UltraGS это порты `BASE+13h` и `BASE+1Bh`; программы с зашитыми адресами General Sound `00B3h` и `00BBh` работают после `UGCFG GSALIAS ON`, когда карта отвечает и по этим адресам.

В байте состояния важны два флага. **CB** (бит 0) поднимается, когда записана команда, и снимается, когда карта её приняла. **DB** (бит 7) показывает, что в защёлке данных лежит байт: после записи программой он установлен, пока карта байт не заберёт; после того как карта положила ответ, он установлен, пока программа ответ не прочитает. Остальные биты состояния равны 1, поэтому проверяют именно эти два бита, а не весь байт.

Обмен одной командой состоит из коротких шагов. В таблице команд они обозначены так:

Шаг	Что делает программа
SD	Записывает байт в DATA
WD	Ждёт, пока карта заберёт байт из DATA (DB = 0)
SC	Записывает код команды в COMMAND
WC	Ждёт приёма команды (CB = 0)
GD	Читает байт из DATA
WN	Ждёт появления следующего байта ответа (DB = 1)

Порядок всегда одинаков. Если у команды есть **предварительный байт** (pre), его записывают в DATA до кода команды, без ожидания: карта заберёт его вместе с командой. Затем SC и WC. Если у команды есть **последующие байты** (post), каждый передают парой SD, WD. Первый байт ответа читают сразу после WC; каждый следующий - после WN. У многих команд ответ необязателен: в защёлке лежит прежнее значение изменённого параметра, и его можно не читать.

Новая команда отменяет недочитанный ответ и незаконченные аргументы предыдущей. Исключение - состояние загрузки (между 30h/38h/3Eh и D2h): в нём карта принимает только байты файла и команду D2h, а прочие команды подтверждает и игнорирует.

Об ошибке команда GS не сообщает: её код остаётся в регистре GS_ERROR (`BASE+07h`, читается также командой F0h) до явного сброса, а в X_STATUS поднимается бит GSERR. После важных команд, например запуска модуля, проверяйте этот регистр.

Библиотечная `UG_GS_COMMAND` выполняет всю последовательность по девятибайтовому описанию: код команды, признак и значение pre, число и адрес post-байтов, число и адрес байтов ответа. Между шагами окно ISA закрывается, а каждое ожидание ограничено тайм-аутом. Короткая обёртка из примера принимает параметры в регистрах:

```
; A = команда GS, B = 1 если есть предварительный байт, C = этот байт,
; E = сколько байт ответа прочитать в gs_reply. Библиотека выполняет
; весь обмен флагами CB/DB и закрывает окно ISA после каждого шага.
gs_command:
    ld (gs_desc),a          ; команда
    ld a,b
    ld (gs_desc+1),a        ; есть ли предварительный байт
    ld a,c
    ld (gs_desc+2),a        ; сам предварительный байт
    ld a,e
    ld (gs_desc+6),a        ; сколько байт ответа прочитать
    ld hl,gs_desc
    jp UG_GS_COMMAND

gs_desc:
    DB 0                    ; код команды GS
    DB 0                    ; has_pre: 0 или 1
    DB 0                    ; pre: байт, который уходит до команды
    DB 0                    ; post_count: байтов после команды нет
    DW 0                    ; адрес байтов после команды
    DB 0                    ; число байтов ответа
    DW gs_reply             ; куда положить ответ
```

9.2. Таблица команд

Коды в таблице шестнадцатеричные и относятся к регистру COMMAND; они не совпадают с номерами команд расширенного API. Колонка «pre; post» перечисляет предварительный и последующие байты, колонка «out» - ответ: число байтов, либо в квадратных скобках необязательное прежнее значение. В столбце назначения приведён символ из `ug_wire.inc` (в коде к нему добавляется префикс `UG_GS_`, например `UG_GS_GS_LOAD_MODULE`) и последовательность обмена в обозначениях 9.1.

GS hex	pre; post	out	Назначение и влияние
00	-	-	GS_RESET_FLAGS. Вне LOAD: сбросить CB/DB и незавершённые legacy аргументы; готовые объекты и музыку сохранить. Пример обмена: SC 00h, WC.
08	-	-	GS_NOP. No-op, квитировать CB; не отменять LOAD, не очищать GS_ERROR. Пример обмена: SC 08h, WC.
20	-	3 LE	GS_POOL_SIZE. Объём пула PSRAM в байтах. Пример обмена: SC 20h, WC, GD; WN/GD × 2.
21	-	3 LE	GS_POOL_FREE. Свободные байты пула; это 21h, несмотря на опечатку 20h в старом руководстве. Пример обмена: SC 21h, WC, GD; WN/GD × 2.
23	-	1	GS_POOL_PAGES. `min(pool_bytes/32768,255)`; насыщение необходимо для 8 МиБ. Пример обмена: SC 23h, WC, GD.
2A	volume; -	[old]	GS_MUSIC_MASTER. Music master 0…64. Пример обмена: SD(pre), SC 2Ah, WC, GD(out), необязательно.
2B	volume; -	[old]	GS_FX_MASTER. FX master 0…64. Пример обмена: SD(pre), SC 2Bh, WC, GD(out), необязательно.
2E	fx; -	[old]	GS_SELECT_FX. Выбрать текущий legacy FX; 0 выбирает последний загруженный. Пример обмена: SD(pre), SC 2Eh, WC, GD(out), необязательно.
30	-	[module]	GS_LOAD_MODULE. Начать загрузку MOD/S3M; формат определяется при завершении D2. Пример обмена: SC 30h, WC, SD/WD для каждого байта файла, SC D2h, WC, GD(handle).
31	module; -	[module]	GS_PLAY_MODULE. Играть с начала: MOD speed=6/BPM=125, S3M speed/BPM из заголовка; music gain=64, master сохранить. Пример обмена: SD(pre), SC 31h, WC, GD(out), необязательно.

GS hex	pre; post	out	Назначение и влияние
32	-	[module]	GS_PAUSE_MODULE. Приостановить текущий MOD, сохранить позицию; FX продолжаются; 0 без MOD. Пример обмена: SC 32h, WC, GD(out), необязательно.
33	-	[module]	GS_RESUME_MODULE. Продолжить MOD; без MOD - no-op с выходом 0, при уже играющем MOD - идемпотентно. Пример обмена: SC 33h, WC, GD(out), необязательно.
35	volume; -	[old]	GS_MUSIC_GAIN. Music gain 0…64, независимо от master. Пример обмена: SD(pre), SC 35h, WC, GD(out), необязательно.
36	-	[FFh]	GS_OUT_FF. Записать FFh в выходную защёлку, без звукового эффекта. Пример обмена: SC 36h, WC, GD.
37	-	-	GS_FREE_LEGACY. Освободить все legacy-owned MOD/FX и сбросить их выбор; независимые EXT объекты сохранить. Пример обмена: SC 37h, WC.
38	-	[fx]	GS_LOAD_FX. Начать загрузку FX U8 mono. Пример обмена: SC 38h, WC, SD/WD для каждого байта файла, SC D2h, WC, GD(handle).
39	fx; -	-	GS_PLAY_FX_SEEK. Запустить FX по seek masks и priority. Пример обмена: SD(pre), SC 39h, WC.
3A	mask; -	-	GS_STOP_FX_MASK. Остановить legacy FX: bit7→GS0, bit6→GS1, bit5→GS2, bit4→GS3; FFh останавливает все legacy FX, не EXT dedicated lanes. Пример обмена: SD(pre), SC 3Ah, WC.
3D	volume; -	[old]	GS_FX_GAIN. FX gain 0…64. Пример обмена: SD(pre), SC 3Dh, WC, GD(out), необязательно.
3E	sign; -	[fx]	GS_LOAD_FX_SIGNED. Загрузка FX: **0=S8, 1=U8** по ROM; прочее ошибка. Пример обмена: SD(pre), SC 3Eh, WC, SD/WD для каждого байта файла, SC D2h, WC, GD(handle).
40	note; -	[old]	GS_FX_NOTE. Нота текущего FX, 0…95. Пример обмена: SD(pre), SC 40h, WC, GD(out), необязательно.
41	volume; -	[old]	GS_FX_VOLUME. Громкость текущего FX, 0…64. Пример обмена: SD(pre), SC 41h, WC, GD(out), необязательно.
42	fine; -	[old]	GS_FX_FINETUNE. Finetune текущего FX, правило ниже. Пример обмена: SD(pre), SC 42h, WC, GD(out), необязательно.
45	priority; -	[old]	GS_FX_PRIORITY. Приоритет текущего FX, 1…254. Пример обмена: SD(pre), SC 45h, WC, GD(out), необязательно.
46	mask; -	[old]	GS_SEEK_FIRST. SeekFirst, младшие четыре бита в GS-нумерации. Пример обмена: SD(pre), SC 46h, WC, GD(out), необязательно.
47	mask; -	[old]	GS_SEEK_LAST. SeekLast, младшие четыре бита в GS-нумерации. Пример обмена: SD(pre), SC 47h, WC, GD(out), необязательно.
48	loop_start.lo; mid,hi	-	GS_LOOP_START. Начало петли U8/S8 в байтах; hi=FFh отключает петлю. Пример обмена: SD(pre), SC 48h, WC, SD/WD × 2.
49	loop_end.lo; mid,hi	-	GS_LOOP_END. Исключительная правая граница петли в байтах. Пример обмена: SD(pre), SC 49h, WC, SD/WD × 2.
60	-	1	GS_ORDER. Текущий order; 0 без MOD. Пример обмена: SC 60h, WC, GD.
61	-	1	GS_ROW. Текущая row; 0 без MOD. Пример обмена: SC 61h, WC, GD.
62	-	1	GS_ORDER_ROW. `((order & 3)<<6) | row`; 0 без MOD. Пример обмена: SC 62h, WC, GD.
63	-	4	GS_NOTES. Ноты MOD 0,1,2,3 в порядке CHANS и флаги изменения, не DAC-порядок. Пример обмена: SC 63h, WC, GD; WN/GD × 3.

GS hex	pre; post	out	Назначение и влияние
64	-	4	GS_VOLUMES. Громкости MOD 0,1,2,3 в порядке CHANS и флаги изменения. Пример обмена: SC 64h, WC, GD; WN/GD × 3.
65	order; -	-	GS_SEEK_ORDER. Seek к order=N, row=0; сбросить канальные эффекты/speed/BPM как SEEK_MOD. Пример обмена: SD(pre), SC 65h, WC.
66	value; -	-	GS_SPEED_BPM. 1…31=speed; 32…255=BPM; 0=F00 с учётом player_mode для MOD, без действия для S3M. Пример обмена: SD(pre), SC 66h, WC.
67	-	1	GS_SPEED. Текущий speed. Пример обмена: SC 67h, WC, GD.
68	-	1	GS_BPM. Текущий BPM. Пример обмена: SC 68h, WC, GD.
69	-	-	GS_EXTRA_TICK. Один дополнительный MOD tick при любом BUSY; утверждённая native-семантика. Пример обмена: SC 69h, WC.
6A	mode; -	[old]	GS_F00_MODE. Для MOD: 0 останавливает на F00, ненулевое игнорирует F00; S3M не затрагивает. Пример обмена: SD(pre), SC 6Ah, WC, GD(out), необязательно.
6B	необязательный lo; hi	-	GS_RELOOPER. Настройка relooper в словах. Пример обмена: SD(pre), SC 6Bh, WC, SD/WD × 1.
80…83	fx; -	-	GS_FX_CH0. Принудительно запустить FX в GS-канале c=opcode−80h. Пример обмена: SD(pre), SC 80h, WC.
88…8B	fx; note	-	GS_FX_CH_NOTE. То же, с нотой только для этого запуска. Пример обмена: SD(pre), SC 88h, WC, SD/WD × 1.
90…93	fx; volume	-	GS_FX_CH_VOL. То же, с громкостью только для этого запуска. Пример обмена: SD(pre), SC 90h, WC, SD/WD × 1.
98…9B	fx; note,volume	-	GS_FX_CH_NV. То же, с нотой и громкостью только для этого запуска. Пример обмена: SD(pre), SC 98h, WC, SD/WD × 2.
A0…A3	note; -	-	GS_SET_NOTE. Изменить ноту активного FX в GS-канале. Пример обмена: SD(pre), SC A0h, WC.
A8…AB	volume; -	-	GS_SET_VOL. Изменить громкость активного FX в GS-канале. Пример обмена: SD(pre), SC A8h, WC.
D0	-	-	GS_EXT_D0. NORMAL: ack/no-op; POST: прервать ожидание, затем no-op; LOAD: ack+ignore. Не API ошибки. Пример обмена: SC D0h, WC.
D1	-	-	GS_EXT_D1. LOAD: идемпотентный маркер, ack+ignore; NORMAL: no-op; POST: прервать ожидание, затем no-op. Пример обмена: SC D1h, WC.
D2	-	[handle]	GS_LOAD_CLOSE. Закрыть stream, проверить/опубликовать объект; 0 при ошибке. Пример обмена: SC D2h, WC, GD.
F0	-	1	GS_READ_ERROR. Прочитать GS_ERROR, не очищать. Пример обмена: SC F0h, WC, GD.
F3	-	-	GS_WARM_RESET. Warm reset всего хранилища, сохранить 6A/6B; сменить session. Пример обмена: SC F3h, WC.
F4	-	-	GS_COLD_RESET. Cold reset хранилища + quick memory test; 6A=0,6B=0; сменить session. Пример обмена: SC F4h, WC.
F5	-	-	GS_BUSY_ON. Busy On: приостановить автоматическое обслуживание MOD, не немедленный mute;. Пример обмена: SC F5h, WC.
F6	-	-	GS_BUSY_OFF. Busy Off: вернуть автоматическое обслуживание MOD, без накопленных пропущенных тиков. Пример обмена: SC F6h, WC.

9.3. Загрузка и управление модулем

Модуль загружают командой 30h, затем передают байты файла по одному (SD, WD), а в конце посылают D2h. Карта проверяет файл и возвращает номер модуля; ноль означает, что файл отвергнут, и причину можно прочитать командой F0h. Промежутки между байтами ограничены только терпением программы: карта не прерывает загрузку по тайм-ауту, поэтому читать файл с диска порциями между передачами можно. Команда D1h, которой старые программы отмечали начало данных, допускается, но не обязательна. Пример открывает карту, проверяет INFO и CAPS и сбрасывает карту так же, как в главе 2 (`reset_card`, 3.5); программе, которая пользуется только командами GS, тот же тёплый сброс даёт команда F3h.

```
gs_load:
    ld a,UG_GS_GS_LOAD_MODULE ; 30h: начать загрузку модуля
    ld b,0
    ld e,1                    ; ответ: номер модуля GS (1)
    call gs_command
    ret c
    ld a,1
    ld (loading),a            ; с этого момента карта ждёт байты файла
    ld hl,music_data
    ld bc,MUSIC_BYTES
.byte:
    ld a,(hl)
    push hl
    push bc
    call UG_GS_SD             ; записать байт в DATA
    jr c,.failed
    call UG_GS_WD             ; дождаться, пока карта его заберёт
.failed:
    pop bc
    pop hl
    ret c
    inc hl
    dec bc
    ld a,b
    or c
    jr nz,.byte
gs_close:
    ld a,UG_GS_GS_LOAD_CLOSE ; D2h: файл передан целиком
    ld b,0
    ld e,1                    ; ответ: номер модуля, 0 при ошибке
    call gs_command
    ret c
    xor a
    ld (loading),a
    ld a,(gs_reply)
    or a
    jr z,.rejected
    ld (loaded),a
    xor a
    ret
.rejected:
    ld a,UG_ERR_FORMAT        ; карта не приняла файл как MOD
    scf
    ret

gs_play:
    ld a,UG_GS_GS_PLAY_MODULE ; 31h
    ld b,1
    ld c,1                    ; предварительный байт: модуль 1
    ld e,1                    ; ответ: 1 = заиграл
    call gs_command
    ret c
    ld a,(gs_reply)
    dec a
    ret z
    ld a,UG_ERR_STATE
    scf
    ret
```

`UG_GS_SD` и `UG_GS_WD` - те самые шаги SD и WD в виде библиотечных процедур; `music_data` и `MUSIC_BYTES` объявлены так же, как в главе 6. `gs_reply` - буфер в памяти Z80, куда `gs_command` кладёт

байты ответа. Флаг `loading` отмечает, что карта ждёт байты файла, `loaded` - что модуль принят; оба нужны общему выходу в конце раздела. 31h с предварительным байтом 1 запускает модуль с начала со стандартным темпом (speed 6, BPM 125); ответ 1 подтверждает запуск. 32h приостанавливает модуль с сохранением позиции, 33h продолжает; при MOD эффекты GS продолжают звучать. Полный сценарий примера:

```
play_lesson:
    call gs_load             ; 30h, байты файла, D2h
    ret c
    call gs_play             ; 31h: модуль 1 с начала
    ret c
    ld a,1
    call wait_seconds
    ret c
    ld a,UG_GS_GS_PAUSE_MODULE ; 32h: пауза с сохранением позиции
    ld b,0
    ld e,0
    call gs_command
    ret c
    ld a,1
    call wait_seconds        ; секунда тишины
    ret c
    ld a,UG_GS_GS_RESUME_MODULE ; 33h: продолжить с того же места
    ld b,0
    ld e,0
    call gs_command
    ret c
    ld a,1
    jp wait_seconds
```

Текущее положение в песне читают командами 60h (order), 61h (row) и 62h, ноты и громкости первых четырёх каналов - 63h и 64h, темп - 67h и 68h; 65h переходит к нужной позиции, 66h меняет speed или BPM. 2Ah и 35h задают мастер-громкость и громкость музыки 0..64.

Очистка в программе GS - команда 37h: она удаляет модуль и эффекты, загруженные командами GS, и останавливает их звук. Объекты, созданные через расширенный API, она не затрагивает. Если загрузка была прервана ошибкой, сначала нужно закрыть её командой D2h - в состоянии загрузки 37h будет проигнорирована:

```
; Общий выход. Незаконченную загрузку сначала закрывают D2h: в состоянии
; LOAD карта игнорирует другие команды. 37h удаляет модуль и эффекты,
; загруженные командами GS, и останавливает их звук.
cleanup:
    ld a,(loading)
    or a
    call nz,gs_close
    ld a,(loaded)
    or a
    ret z
    ld a,UG_GS_GS_FREE_LEGACY ; 37h
    ld b,0
    ld e,0
    jp gs_command
```

Запустите `GSMOD0`: ожидается секунда музыки, секунда паузы, ещё секунда музыки и сообщение `GSMOD: module loaded by GS commands, played, paused, resumed, freed.`

9.4. Эффекты GS

Эффект загружают так же, как модуль, но командой 38h для беззнаковых восьмибитных отсчётов или 3Eh с предварительным байтом формата: 0 для знаковых (S8), 1 для беззнаковых (U8). Карта хранит до 60 эффектов GS с номерами 1..60. Команда 2Eh выбирает текущий эффект; команды 40h..49h задают его параметры по умолчанию: ноту 0..95 (нота 60 соответствует периоду 214), громкость 0..64, подстройку, приоритет 1..254, маски поиска канала и границы петли в байтах (три байта: pre и два post).

У GS четыре выхода эффектов GS0..GS3, соответствующие каналам MOD 0, 3, 1, 2. Команда 39h запускает текущий эффект в свободном канале по маскам поиска, вытесняя при необходимости эффект с меньшим

приоритетом. Команды 80h..83h запускают эффект прямо в заданном канале; 88h..9Bh делают то же с нотой и/или громкостью только для этого запуска; A0h..ABh меняют ноту и громкость уже звучащего эффекта; 3Ah останавливает эффекты по маске (бит 7 - GS0, бит 6 - GS1, бит 5 - GS2, бит 4 - GS3). Пока эффект GS звучит в канале, он временно заменяет инструмент модуля в этом канале - так работала настоящая GS. Эффект расширенного API (PLAY_FX) инструмент не заменяет; смешивать оба способа в одной программе можно.

Однобайтовые номера GS (модуль 1, эффекты 1..60) - не те же числа, что handle расширенного API. Команда LEGACY_INFO возвращает настоящие handle текущего модуля, эффекта и незавершённой загрузки, если программе нужно обратиться к ним через расширенный API, например прочитать OBJECT_INFO.

9.5. Отличия от General Sound и NeoGS

Вопрос	Как ведёт себя UltraGS
Команды Z80 внутри платы: 10h..1Fh, прямой DAC 01h..07h, Covox, POKE/PEEK/CALL	Не исполняются: GS_ERROR = 01h. Внутренней RAM, MP3 и SD-карты нет
08h	Пустая команда с подтверждением, а не сброс флагов; сброс флагов - 00h
21h	Свободная память; в старом описании команда ошибочно повторяла номер 20h
23h	Число 32-килобайтных страниц, не больше 255; для 8 МиБ читайте 20h
3Eh	0 - знаковые отсчёты, 1 - беззнаковые, как в ROM GS; старое руководство утверждало обратное
32h	Пауза с сохранением позиции и ресурсов, а не выгрузка модуля
63h, 64h	Ноты и громкости в естественном порядке каналов 0,1,2,3; бит 7 означает «не менялось с прошлого чтения»
6Ah	0 - останавливаться на F00 с сохранением позиции, иначе игнорировать F00
F3h, F4h	Тёплый и холодный сброс всей карты со сменой сеанса - то же, что RESET_STORE уровней 0 и 1 (3.5); выполняются в начале работы, а не при выходе
F5h, F6h, 69h	Приостановка и возобновление автоматических тиков модуля и один ручной тик; звук не глушится
Неподдержанные коды	GS_ERROR = 01h, без зависания; состояние CB снимается

10. AdLib OPL2

10.1. Доступ к синтезатору

Карта всегда отвечает на портах AdLib: `0388h` принимает номер регистра и отдаёт байт состояния, `0389h` принимает значение регистра. Бит 13 поля `features` в CAPS подтверждает поддержку. Девять FM-каналов синтезатора не расходуют голоса PCM, а общие уровни SET_MIX на них не действуют.

Запись в OPL2 - два обращения: номер регистра в `0388h`, затем значение в `0389h`. Чипу нужно время: не меньше 3,3 мкс после номера регистра и не меньше 23 мкс после значения. Паузу обычно заполняют несколькими чтениями порта состояния. Библиотечной команды для OPL2 нет - программа обращается к портам сама, открывая окно ISA вызовом `UG_ISA_OPEN` (A = слот, HL = 0388h) и закрывая его `UG_ISA_CLOSE`. Пока окно открыто, порт с номером N доступен как ячейка памяти C000h + N:

```
OPL_ISA_BASE EQU 0x0388      ; порты OPL2: 0388h адрес/статус, 0389h данные
OPL_ADDRESS  EQU 0xC388      ; те же порты в окне ISA: C000h + номер порта
OPL_DATA     EQU 0xC389

; A = номер регистра OPL2, E = значение. Между записью адреса и данных
; чипу нужно не меньше 3,3 мкс, после данных - не меньше 23 мкс; паузы
; заполняются чтением статуса. Окно ISA открыто только на время записи.
opl_write:
    ld (opl_reg),a
    ld a,e
    ld (opl_value),a
    ld a,DOC_SLOT
    ld hl,OPL_ISA_BASE
    call UG_ISA_OPEN
    ret c
    ld a,(opl_reg)
    ld (OPL_ADDRESS),a
    ld b,6
.address_delay:
    ld a,(OPL_ADDRESS)
    djnz .address_delay
    ld a,(opl_value)
    ld (OPL_DATA),a
    ld b,36
.data_delay:
    ld a,(OPL_ADDRESS)
    djnz .data_delay
    jp UG_ISA_CLOSE

; HL = таблица пар "регистр, значение", конец - байт FFh.
write_registers:
    ld a,(hl)
    cp 0xFF
    jr z,.done
    inc hl
    ld e,(hl)
    inc hl
    push hl
    call opl_write
    pop hl
    ret c
    jr write_registers
.done:
    xor a
    ret
```

`opl_reg` и `opl_value` - две однобайтовые переменные: `UG_ISA_OPEN` меняет A и HL, поэтому номер регистра и значение сохраняются до открытия окна. `write_registers` записывает таблицу пар до байта FFh; так в 10.2 задаётся инструмент.

Чтение `0389h` возвращает FFh: регистры синтезатора не читаются. Байт состояния содержит 06h в младших битах, бит 6 - переполнение таймера 1, бит 5 - таймера 2, бит 7 - общий признак. Таймеры задаются регистрами 02h и 03h (период (256−значение)×80 мкс и ×320 мкс) и управляются регистром 04h: биты 0 и 1 запускают таймеры, биты 6 и 5 маскируют их, бит 7 сбрасывает флаги. Так программы

определяют наличие AdLib: сброс флагов, запуск таймера 1 с периодом 80 мкс и проверка, что через 100 мкс старшие биты состояния стали C0h.

Ограничение: если в настройках карты включён дополнительный блок портов BULK alias и он лежит вне 0380h-039Fh, чтение состояния `0388h` надёжно только при выключенном Turbo (сборка с `UG_ISA_TURBO_OFF`, см. 1.4). С выключенным alias (настройка по умолчанию) ограничения нет; запись регистров его не имеет в любом случае.

10.2. Инструмент и нота

Каждый из девяти каналов OPL2 образован двумя операторами - генераторами с собственной огибающей. Один модулирует другой (FM-связь), либо их звуки складываются. Тембр задают регистры операторов, высоту ноты - регистры канала. Номера регистров оператора получаются сложением базового номера со смещением оператора; для канала 0 смещения равны 0 (модулятор) и 3 (носитель).

Регистр	Назначение
01h	Бит 5 разрешает выбор формы волны регистрами E0h
20h + оператор	Множитель частоты, вибрато, тремоло, удержание уровня
40h + оператор	Ослабление уровня: у носителя это громкость ноты
60h + оператор	Скорость атаки (старшие 4 бита) и спада
80h + оператор	Уровень удержания и скорость затухания после отпускания
E0h + оператор	Форма волны 0..3
C0h + канал	Обратная связь модулятора и способ связи операторов
A0h + канал	Младшие 8 бит f-number (высоты ноты)
B0h + канал	Бит 5 - клавиша нажата; биты 2..4 - октава; биты 0..1 - старшие биты f-number
BDh	Глубина вибрато и тремоло; бит 5 включает ударные на каналах 6..8

Высота ноты задаётся парой «октава, f-number»: частота = f-number × 49716 / 2^(20 − октава). Для ля первой октавы (440 Гц) при октаве 4 получается f-number 580 = 244h: в A0h записывают 44h, в B0h - 32h (нажатая клавиша, октава 4, старшие биты 2). Отпускание клавиши - запись тех же битов без бита 5: 12h. После отпускания звук затухает со скоростью, заданной регистром 80h.

Пример fm.asm настраивает канал 0 и играет эту ноту одну секунду. Инструмент задан таблицей пар «регистр, значение»:

```
instrument:
    DB 0x01,0x20            ; разрешить выбор формы волны
    DB 0x20,0x21            ; модулятор: множитель частоты 1, удержание
    DB 0x23,0x21            ; носитель: то же
    DB 0x40,0x18            ; уровень модулятора: ослабить на 18 дБ
    DB 0x43,0x00            ; носитель на полной громкости
    DB 0x60,0xF3            ; быстрая атака, умеренный спад
    DB 0x63,0xF3
    DB 0x80,0x35            ; уровень удержания и скорость затухания
    DB 0x83,0x35
    DB 0xE0,0x00            ; синус у обоих операторов
    DB 0xE3,0x00
    DB 0xC0,0x00            ; FM-связь операторов, без обратной связи
    DB 0xFF

NOTE_FNUM_LO EQU 0x44       ; A-4 (440 Гц): f-number 244h при октаве 4
NOTE_KEY_ON  EQU 0x32       ; бит 5 = клавиша нажата, октава 4, f-number[9:8]=2
NOTE_KEY_OFF EQU 0x12       ; те же биты без бита 5

play_lesson:
    call silence_all        ; снять старые ноты, мелодический режим
    ret c
    ld hl,instrument
```

```
    call write_registers     ; тембр канала 0
    ret c
    ld a,0xA0
    ld e,NOTE_FNUM_LO        ; младшие 8 бит высоты ноты
    call opl_write
    ret c
    ld a,1
    ld (sounding),a
    ld a,0xB0
    ld e,NOTE_KEY_ON         ; старшие биты, октава и нажатие клавиши
    call opl_write
    ret c
    ld a,1
    call wait_seconds        ; нота звучит
    ret c
    ld a,0xB0
    ld e,NOTE_KEY_OFF        ; клавиша отпущена: начинается затухание
    call opl_write
    ret c
    xor a
    ld (sounding),a
    ld a,1
    jp wait_seconds          ; слышно затухание
```

Перед первой нотой `silence_all` снимает клавиши всех девяти каналов и выбирает мелодический режим (BDh = 0): предыдущая программа могла оставить синтезатор в любом состоянии:

```
; Снять клавиши всех девяти каналов и выбрать мелодический режим.
silence_all:
    ld a,0xB0
.channel:
    push af
    ld e,0
    call opl_write
    pop bc
    ret c
    ld a,b
    inc a
    cp 0xB9
    jr nz,.channel
    ld a,0xBD
    ld e,0
    jp opl_write
```

У OPL2 нет понятия владельца: карта не знает, чья нота звучит в канале. Поэтому программа сама распределяет каналы, а при выходе отпускает ровно те клавиши, которые нажала. Флаг `sounding` отмечает, что клавиша канала 0 нажата; общий выход по нему записывает `NOTE_KEY_OFF`:

```
; Общий выход: отпустить свою ноту, если она ещё нажата. Другие каналы
; программа не трогает: у OPL2 нет учёта владельцев, ноты снимает тот,
; кто их нажал.
cleanup:
    ld a,(sounding)
    or a
    ret z
    ld a,0xB0
    ld e,NOTE_KEY_OFF
    jp opl_write
```

Запустите `FM0` (или `FM1` для второго слота): ожидается нота, её затухание и сообщение `FM: note played and released`.

Синтезатор работает одновременно с PCM: музыка MOD и эффекты не затрагивают регистры OPL2, а остановка музыки не отпускает клавиши FM. Громкость FM регулируется только уровнями операторов (регистры 40h) и общей настройкой карты.

11. Приложения

11.1. Порты, биты состояния и возможностей

Адреса ячеек ниже рассчитаны для BASE = 02A0h. Портам ISA соответствуют ячейки окна C000h + номер порта; библиотека открывает это окно только на время обращения.

Смещ.	ISA / Z80	Регистр	Чтение	Запись
00	02A0 / C2A0	BULK_DATA	bulk_byte	bulk_byte
01	02A1 / C2A1	X_STATUS	status	bad_register
02	02A2 / C2A2	X_CONTROL	zero	control
03	02A3 / C2A3	X_REQUEST	response_byte	request_byte
04	02A4 / C2A4	SNAP_INDEX	index	index
05	02A5 / C2A5	SNAP_DATA	snap_byte	bad_register
06	02A6 / C2A6	X_ERROR	xerror	bad_register
07	02A7 / C2A7	GS_ERROR	gserror	bad_register
08	02A8 / C2A8	API_MAJOR	const	bad_register
09	02A9 / C2A9	API_MINOR	const	bad_register
0A..12	02AA / C2AA	RSVD_0A	zero	bad_register
13	02B3 / C2B3	GS_DATA	gs_data_out	gs_data_in
14..1A	02B4 / C2B4	RSVD_14	zero	bad_register
1B	02BB / C2BB	GS_CMDSTAT	gs_status	gs_command
1C..1F	02BC / C2BC	RSVD_1C	zero	bad_register
-	01D0..01D2 / C1D0..C1D2	CONFIG	selector/data	unlock/select/data
-	00B3 / C0B3	GSALIAS DATA	GS out	GS in
-	00BB / C0BB	GSALIAS CMD/STATUS	status	command
-	0388 / C388	OPL2	status	address
-	0389 / C389	OPL2 data	FFh	value

X_STATUS бит	Название
0	READY
1	BUSY
2	REPLY
3	WRITE_GRANT
4	READ_GRANT
5	XERR
6	GSERR
7	AUDIO

Порты карты рассчитаны на обращение при включённом Turbo. Исключений два. Порты настройки 01D0h-01D2h доступны только при выключенном Turbo; библиотека к ним не обращается, их использует `UGCFG`. Чтение состояния OPL2 по адресу 0388h надёжно при включённом Turbo, пока в настройках карты не включён BULK alias вне 0380h-039Fh; с таким alias состояние читают при выключенном Turbo (см. 1.4 и 10.1).

CAPS.features бит	Значение
0	Музыкальные команды GS
1	Передача данных через порт GS DATA
2	Передача данных блоками (BULK)
3	Включён дополнительный блок портов BULK alias
4	Зарезервирован
5	Согласованные снимки состояния
6	Проверка CRC32
7	MEM_TEST
8	Независимые эффекты PLAY_FX
9	Включены адреса GS 00B3h/00BBh (GSALIAS)
10	Stereo-эффекты
11	Поток PCM
12	Явная политика F00 в PLAY_MOD
13	Синтезатор OPL2

CAPS.formats: бит 0 RAW 8 бит, бит 1 RAW 16 бит, бит 2 WAV PCM, бит 3 MOD с четырьмя каналами (M.K., M!K!, FLT4), бит 4 8CHN, бит 5 FLT8, бит 6 MOD с 1..32 каналами, бит 7 S3M, бит 8 WAV IMA ADPCM. Неизвестные биты не используйте.

11.2. Коды ошибок

Код hex	Ошибка	Действие
00	OK	Операция завершена; проверить длину ответа.
01	BAD_OPCODE	Проверить номер команды и CAPS; зарезервированные номера не использовать.
02	BAD_ARGUMENT	Проверить флаги, reserved, gain и параметры.
03	LENGTH	Исправить длину payload/число данных, сбросить FAULT при необходимости.
04	CRC	Не использовать повреждённый ответ; сверить результат изменяющей команды.
05	BUSY	Дождаться конца операции с ограничением по времени. Незавершённую передачу снимает TRANSPORT_RESET, незакрытую загрузку GS - D2h.
06	STATE	Проверить состояние/ключ snapshot/READY, начать новый snapshot при stale key.
07	HANDLE	Проверить session и принадлежность handle/ID.
08	RANGE	Проверить offset, N, номер страницы, order/row.
09	NO_MEMORY	Проверить free и largest; освободить свои объекты либо уменьшить запрос.
0A	NO_HANDLES	Лимит объектов или номеров; освободить свои объекты. Счётчик номеров обнуляет только RESET_STORE (3.5).
0B	FORMAT	Проверить поддержанный формат и целостность файла.
0C	NO_VOICE	Отложить звук; не останавливать чужую музыку.
0D	TIMEOUT	Закрыть окно, выполнить ограниченное восстановление и сверку состояния.
0E	OVERFLOW	Не передавать лишние байты; проверить исчерпание token/sequence.
0F	UNDERFLOW	Не читать FIFO без REPLY/READ_GRANT; восстановить framing.
10	SHORT_TRANSFER	Додать нельзя после отмены: начать корректный блок заново.
11	PSRAM	Остановить сценарий; сохранить диагностику памяти.
12	PINNED	Остановить свои instances и дождаться снятия pins.
13	UNSUPPORTED_MEMORY	Память платы не поддержана; проверить аппаратную конфигурацию.
14	BAD_REGISTER	Исправить адрес/направление доступа.
15	CANCELLED	Операция отменена; завершить очистку собственных ресурсов.
16	INTERNAL	Сохранить диагностику; не продолжать изменение store.

Регистр GS_ERROR использует те же коды, кроме двух: нехватка памяти - 10h, нехватка номеров - 11h; ошибка памяти карты отображается как E1h, неполная передача - как E0h. Команда F0h читает регистр, не очищая его; очищают 00h вне загрузки, F3h/F4h и `UG_XC_CLEAR_ERRORS`. Коды выхода утилит DSS с кодами API не совпадают: например, утилиты печатают TIMEOUT как code 8, ESC как code 10.

11.3. Утилиты

Утилита	Назначение и типичная команда
UGCFG	`INFO`, `INFO /DIAG`, `ULTRAGS`, `PORTS`, `GSALIAS ON`, `SAVE`, `VERIFY` - режим и адреса карты
UGMEM	`INFO`, `LIST`, `TEST QUICK`, `TEST FULL`, `RESET WARM`, `RESET COLD`, `ROUNDTRIP N /VERIFY`, `FREE handle` - память карты и её сброс вручную
UGLOAD	`файл /TYPE:RAW|WAV|MOD|S3M` - загрузить файл и напечатать handle
UGDUMP	`handle файл /VERIFY` - выгрузить объект побайтно
UGPLAY	`файл /WAIT`, `HANDLE h`, `FXHANDLE h`, `PAUSE`, `RESUME`, `STOP`, `STREAM файл`
UGMON	`/COUNT:N` - снимки состояния и голоса
UGTEST	`PCM`, `FX`, `MIX`, `GS`, `MOD4`, `MOD8`, `FLT8` - проверочные сценарии
UGOPL	`TEST`, `MIX /FX:MONO|STEREO` - диагностика OPL2
UGPF, UGBENCH, UGDIAG	Ускоренная подача, измерения обмена, диагностика

Синтаксис и доступные ключи печатает `/?`. Номера handle и запусков в выводе утилит относятся к текущему сеансу карты. UGLOAD оставляет объект для следующих программ; UGPLAY без `/WAIT` может оставить звук играть после возврата в DSS.

11.4. Словарь

Термин	Значение
Объект (object)	Данные, хранящиеся в памяти карты, с метаданными
Handle	Двухбайтовый номер объекта в текущем сеансе
Запуск (instance)	Отдельное воспроизведение объекта; play_id для основного звука, instance_id для эффекта
Голос (voice)	Один из 32 физических каналов микшера PCM
Основной звук (foreground)	Единственный MOD/S3M, PCM или поток; эффекты играют поверх
Эффект (FX)	Независимый запуск PCM-объекта командой PLAY_FX или командами GS
Pin	Признак занятости объекта запуском или передачей; запрещает FREE
SEAL	Проверка загруженного объекта и перевод его в READY
Кадр (frame)	Отсчёты всех каналов в один момент времени; в описании протокола - также один запрос или ответ
Сеанс (session)	Идентификатор состояния хранилища; меняется при сбросе
Снимок (snapshot)	Согласованная копия состояния карты для чтения страницами

11.5. Источники

- General Sound v1.04, редакция 006, с дополнениями ROM 1.05: описание команд GS, которым следует глава 9.
- NeoGS. Руководство программиста, version 0.1.A: порядок изложения «характеристики - устройство - интерфейс - команды - примеры».
- T3 UltraGS: требования и принятые решения; при расхождении с этим руководством действует T3.
- Wire schema и библиотека DSS: форматы команд и реализация обмена, с которыми сверены таблицы приложений.

11.6. Форматы команд UltraGS API 1.0

Смещения считаются от начала полезной нагрузки запроса или ответа. Все многобайтовые числа little-endian, без выравнивания. u8/u16/u32/u64 - беззнаковые числа, s8 - знаковое, `byte[N]` - N байт, `FXREC[]` и `VOICEREC[]` - записи переменного количества. Структуры PCM_DESC, PAGE_HEADER, VOICEREC и FXREC описаны в 11.7. Успешный ответ имеет статус 0; при ошибке данных ответа нет.

00h INFO

Определяет версию и состояние карты. До ALLOC/PLAY проверьте UGS1, API 1.0, VLS1 и boot_state=0; сохраните session. Лишние байты запроса дают BAD_OPCODE. Команда доступна даже при ошибке памяти.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: INFO (40 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	magic	byte[4]	4	значение UGS1
4	api_major	u8	1	значение 1
5	api_minor	u8	1	значение 0
6	mode	u8	1	значение 8
7	boot_state	u8	1	-
8	fw_version	u32	4	-
12	session	u64	8	-
20	build_id	byte[16]	16	-
36	layout	byte[4]	4	значение VLS1

Разрешена во время передачи блока.

01h CAPS

Возвращает доступные функции, форматы и пределы. Проверяйте нужные биты перед использованием функции. music_limit и fx_limit относятся к общему пулу и не складываются. Во время передачи блока возвращает BUSY.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: CAPS (44 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	features	u32	4	-
4	formats	u32	4	-
8	output_hz	u32	4	-
12	max_block	u16	2	значение 1024
14	max_payload	u16	2	значение 128
16	objects	u16	2	значение 128

Смещ.	Поле	Тип	Байт	Примечание
18	playlist_entries	u16	2	Зарезервировано
20	total_voices	u16	2	-
22	music_limit	u16	2	-
24	fx_limit	u16	2	-
26	gs_profile	u8	1	-
27	pool_kind	u8	1	значение 1
28	base	u16	2	-
30	alias	u16	2	-
32	pcm_rate_min	u32	4	значение 8000
36	pcm_rate_max	u32	4	значение 48000
40	voice_record_size	u8	1	значение 20
41	fx_record_size	u8	1	значение 16
42	snapshot_size	u16	2	значение 64

02h MEM_INFO

Возвращает размеры памяти в байтах и число объектов. Для одного ALLOC нужен непрерывный участок largest. Не меняет память и не останавливает звук.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: MEM_INFO (20 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	physical	u32	4	-
4	pool	u32	4	-
8	free	u32	4	-
12	largest	u32	4	-
16	objects	u16	2	-
18	limit	u16	2	-

Разрешена во время передачи блока.

03h STATS

Читает 16 счётчиков u32. max_render_us и max_command_us измеряются в микросекундах; rx_high_water и tx_high_water - пиковое заполнение FIFO в байтах. Остальные поля считают операции, байты, блоки или отсчёты по имени. Для наблюдения за своим сценарием сравнивайте значения до и после него.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: STATS (64 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	isa_reads	u32	4	-
4	isa_writes	u32	4	-
8	upload_bytes	u32	4	-
12	download_bytes	u32	4	-
16	write_blocks	u32	4	-
20	read_blocks	u32	4	-
24	crc_errors	u32	4	-
28	protocol_errors	u32	4	-
32	transfer_timeouts	u32	4	-
36	audio_underruns	u32	4	-
40	psram_errors	u32	4	-
44	max_render_us	u32	4	-
48	max_command_us	u32	4	-
52	rx_high_water	u32	4	-
56	tx_high_water	u32	4	-
60	clip_samples	u32	4	-

Разрешена во время передачи блока.

04h CLEAR_STATS

Обнуляет общую статистику. Звук и объекты сохраняются. Команда влияет и на измерения других наблюдателей; для обычного мониторинга предпочтительнее разность двух STATS.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK.

Нет полезных данных (0 байт).

05h MEM_TEST

mode=0: короткая проверка, mode=1: полная. Требует пустого хранилища; при занятых ресурсах BUSY. При ошибке PSRAM результат теста всё равно содержит tested, first_bad и expected/actual. На успехе first_bad=FFFFFFFFh.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	mode	u8	1	-

Ответ при OK: MEM_TEST_RESP (14 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	tested	u32	4	-
4	first_bad	u32	4	-
8	expected	u8	1	-
9	actual	u8	1	-
10	elapsed_ms	u32	4	-

06h RESET_STORE

Сбрасывает хранилище карты целиком. Ключ 55475253h - это буквы «UGRS» (UltraGS Reset Store), прочитанные как число u32; в запросе он занимает байты 53h, 52h, 47h, 55h. Ключ защищает от случайного вызова: с другим значением команда возвращает BAD_ARGUMENT. level=0 удаляет все объекты, останавливает воспроизведение и незавершённые передачи, настройки GS 6Ah и 6Bh сохраняет; level=1 дополнительно сбрасывает эти настройки и выполняет короткую проверку памяти. Ответ содержит новую session; все прежние handles и номера запусков недействительны. Так программа начинает работу с чистой карты (3.5); при выходе достаточно освободить свои объекты. При незавершённой передаче или незакрытой загрузке GS возвращает BUSY: первую снимает TRANSPORT_RESET, вторую закрывает D2h.

Запрос: RESET_STORE_REQ (5 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	key	u32	4	значение 0x55475253
4	level	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	session_id	u64	8	-

10h ALLOC

Создаёт UPLOADING-объект kind=0 BLOB, 1 RAW, 2 WAV, 3 MOD или 5 S3M. size задаётся в байтах, положительный. Сохраняйте возвращённый handle; reserve - фактический объём с округлением до 16. NO_MEMORY означает нехватку непрерывной памяти, NO_HANDLES - лимит объектов или номеров.

Запрос: ALLOC_REQ (5 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	kind	u8	1	-
1	size	u32	4	-

Ответ при OK: ALLOC_RESP (6 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	reserved	u32	4	Фактический резерв в байтах, округлён до 16

11h FREE

Удаляет указанный объект. Сначала остановите все собственные запуски, использующие его. PINNED означает, что объект ещё нужен проигрывателю или передаче; повтор допускается после ожидания. Неизвестный handle даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

12h OBJECT_INFO

Читает метаданные объекта: state=0 загрузка, 1 готов, 2 испорчен; pins запрещает удаление. size и uploaded измеряются в байтах. Параметры звука имеют смысл после PCM_CONFIG или успешного разбора SEAL; sample_format: 0 U8, 1 S8, 2 S16LE, 3 IMA ADPCM (тогда bits=4). Неизвестный объект даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK: OBJECT_INFO (38 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	kind	u8	1	-
3	state	u8	1	-
4	pins	u16	2	-
6	size	u32	4	-
10	uploaded	u32	4	-
14	crc	u32	4	-
18	data_offset	u32	4	-
22	data_length	u32	4	-
26	rate	u32	4	-
30	frames	u32	4	-
34	channels	u8	1	-
35	bits	u8	1	-
36	sample_format	u8	1	-
37	mod_channels	u8	1	-

Разрешена во время передачи блока; только для объекта этой передачи.

13h SEAL

Проверяет полностью загруженный объект и переводит его в READY. Для RAW предварительно нужен PCM_CONFIG. flags=0 требует expected_crc=0; flags=1 проверяет CRC всего объекта. Успех возвращает actual_crc. STATE означает неподходящую фазу; FORMAT/CRC - неверное содержимое. Не повторяйте SEAL объекта READY.

Запрос: SEAL_REQ (7 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	flags	u8	1	Бит 0: проверить CRC всего объекта
3	expected_crc	u32	4	-

Ответ при OK: SEAL_RESP (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	actual_crc	u32	4	-

14h PCM_CONFIG

Задаёт описание только RAW в состоянии UPLOADING, без pins. Вложенная структура PCM_DESC приведена в 11.7. Неправильные частота, формат, длина или петля дают FORMAT; другой kind или состояние - STATE.

Запрос: PCM_CONFIG_REQ (26 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	desc	PCM_DESC	24	-

Ответ при OK.

Нет полезных данных (0 байт).

15h OBJECT_ENUM

Обходит handles по возрастанию: первый запрос after_handle=0, следующий - с ранее возвращённым next_handle. Ноль в ответе завершает обход. Перечисление само по себе не доказывает, что объект принадлежит вызывающей программе.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	after_handle	u16	2	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	next_handle	u16	2	-

16h LEGACY_INFO

Возвращает native handles текущих GS MOD, FX и незавершённой загрузки; 0 означает отсутствие. uploaded - число принятых байтов LOAD. Эти u16 handles отличаются от однобайтовых номеров GS.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: LEGACY_INFO_RESP (10 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	module	u16	2	-
2	fx	u16	2	-
4	pending	u16	2	-
6	uploaded	u32	4	-

Разрешена во время передачи блока.

20h WRITE_BEGIN

Начинает запись N=1..1024 байт. Для последовательной загрузки offset=uploaded; BLOB также допускает перезапись уже загруженного диапазона. transport=00h/01h FAST или 80h/81h VERIFIED. Сначала получите token и WRITE_GRANT, затем отправьте данные. STATE/RANGE означают неподходящую фазу или диапазон.

Запрос: WRITE_BEGIN_REQ (9 байт FAST; 13 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	offset	u32	4	-
6	length	u16	2	-
8	transport	u8	1	-
9	crc	u32	4	VERIFIED: полный размер 13 байт; только VERIFIED

Ответ при OK: WRITE_BEGIN_RESP (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-

21h WRITE_END

Передаёт token после всех N байт. Только успешный END подтверждает запись. SHORT_TRANSFER означает недостачу данных, CRC - несовпадение в VERIFIED. После отказа блок не считается подтверждённым; сверяйте OBJECT_INFO перед продолжением.

Запрос: WRITE_END_REQ (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-

Ответ при OK.

Нет полезных данных (0 байт).

Разрешена во время передачи блока.

22h READ_BEGIN

Начинает чтение N=1..1024 байт в пределах uploaded; объект может быть READY и играть. После token дождитесь READ_GRANT. В VERIFIED ответ дополнительно содержит CRC данных. Запрос CRC не содержит. RANGE означает чтение за доступной границей.

Запрос: READ_BEGIN_REQ (9 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	offset	u32	4	-
6	length	u16	2	-
8	transport	u8	1	-

Ответ при OK: READ_BEGIN_RESP (4 байт FAST; 8 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-
4	crc	u32	4	VERIFIED: полный размер 8 байт; только VERIFIED

23h READ_END

Завершает чтение по token. В VERIFIED передайте вычисленную host_crc; в FAST поле отсутствует. Неверный token даёт HANDLE, неполное чтение - SHORT_TRANSFER, несовпадение CRC - CRC. Данные объекта не меняются.

Запрос: READ_END_REQ (4 байт FAST; 8 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-
4	host_crc	u32	4	VERIFIED: полный размер 8 байт; только VERIFIED

Ответ при OK.

Нет полезных данных (0 байт).

Разрешена во время передачи блока.

24h TRANSFER_CANCEL

Отменяет передачу с данным token, освобождая её временный буфер и pin. Ранее завершённые блоки сохраняются. Неизвестный или уже завершённый token даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-

Ответ при OK.

Нет полезных данных (0 байт).

Разрешена во время передачи блока.

30h PLAY_PCM

Запускает READY RAW/WAV как основной звук. repeats=0 повторяет весь объект, 1 играет один раз, 2..65535 задаёт число проходов; внутренняя RAW-петля требует repeats=1. gain=0..64, pan=0..255. Возвращает play_id. При NO_VOICE/BUSY новый запуск не принят; прежний звук сохраняется.

Запрос: PLAY_PCM_REQ (6 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	repeats	u16	2	-
4	gain	u8	1	-
5	pan	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-

31h PLAY_MOD

Запускает READY MOD/S3M с начала и возвращает play_id. flags: 1 LOOP, 2 IGNORE_F00, 4 PAUSE_F00; последние два несовместимы. Для MOD без них действует GS6A; S3M разрешает только LOOP, F00 flags дают BAD_ARGUMENT. Некорректные flags дают BAD_ARGUMENT, нехватка голосов - NO_VOICE, неподходящий объект - STATE/FORMAT.

Запрос: PLAY_MOD_REQ (3 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	flags	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-

32h CONTROL

Управляет основным звуком: action=0 STOP, 1 PAUSE, 2 RESUME. play_id=0 выбирает текущий запуск, поэтому для очистки используйте точный ID. RESUME требует паузы. Нет подходящего активного запуска - STATE. STOP не удаляет обычный объект, но закрывает поток.

Запрос: CONTROL_REQ (3 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-
2	action	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

33h SEEK_MOD

Переводит активный MOD/S3M на order и row=0..63. order меньше длины песни. play_id=0 означает текущий tracker; неверный ID или не tracker дают STATE, неверная позиция - RANGE. Эффекты и темп сбрасываются: MOD на 6/125, S3M на значения заголовка, состояние паузы сохраняется.

Запрос: SEEK_MOD_REQ (5 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-
2	order	u16	2	-
4	row	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

34h SET_MIX

Четыре уровня 0..64: music_master, fx_master, music_gain, fx_gain. Изменяет общие уровни основного звука и FX, включая GS-эффекты; OPL2 не затрагивает. Значение выше 64 даёт BAD_ARGUMENT. Это общая настройка, без ID отдельного запуска.

Запрос: SET_MIX_REQ (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	music_master	u8	1	-
1	fx_master	u8	1	-
2	music_gain	u8	1	-
3	fx_gain	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

35h SET_TEMPO

Меняет текущий tracker: speed=1..31 для MOD, 1..255 для S3M, BPM=32..255. Нет активного tracker - STATE, неверные значения - BAD_ARGUMENT. Последующие команды темпа внутри песни могут снова изменить значения.

Запрос: SET_TEMPO_REQ (2 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	speed	u8	1	-
1	bpm	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

36h PLAY_FX

Создаёт независимый запуск READY RAW/WAV. preferred_voice=FFFFh выбирает голоса автоматически, иначе 0..31. priority=1..254, gain=0..64, pan=0..255. flags: 1 разрешает вытеснять FX меньшего приоритета, 2 повторяет весь объект. Музыка не вытесняется; при NO_VOICE ни один канал эффекта не запускается.

Запрос: PLAY_FX_REQ (8 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	preferred_voice	u16	2	-
4	priority	u8	1	-
5	gain	u8	1	-
6	pan	u8	1	-
7	flags	u8	1	-

Ответ при OK: PLAY_FX_RESP (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	instance_id	u16	2	-
2	voice_count	u16	2	-

37h STOP_FX

Останавливает instance_id эффекта, не удаляя его объект. ID=0 останавливает все native FX; для очистки одного приложения он не подходит. Несуществующий ненулевой ID даёт HANDLE. Музыку и GS direct FX команда не останавливает.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	instance_id	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

38h SET_PAN

Меняет панораму логического tracker-канала, а не физического голоса. channel должен существовать и быть 0..31; иначе RANGE. pan=0..255. Нет активного tracker - STATE. Настройку могут заменить PLAY, SEEK и команды самого MOD.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	logical_channel	u16	2	-
2	pan	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

39h CONTROL_FX

Управляет одним ненулевым instance_id: action=0 STOP, 1 PAUSE, 2 RESUME. При паузе голоса и позиция сохраняются. Нет такого FX - HANDLE, RESUME без паузы - STATE. Основной звук и другие FX продолжаются.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	fx_id	u16	2	-
2	action	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

40h QUEUE_CLEAR (резерв)

Зарезервирована. Карта отвечает BAD_OPCODE.

41h QUEUE_ADD (резерв)

Зарезервирована. Карта отвечает BAD_OPCODE.

42h QUEUE_PLAY (резерв)

Зарезервирована. Карта отвечает BAD_OPCODE.

43h QUEUE_INFO (резерв)

Зарезервирована. Карта отвечает BAD_OPCODE.

44h STREAM_CREATE

Создаёт единственное PCM-кольцо. rate=8000..48000, channels=1/2, bits=8/16, формат U8/S8/S16LE; reserved=0. Размеры в байтах, кратные размеру PCM-кадра; buffer>0, 0<prefill<=buffer, 0<=low<prefill. Второе кольцо даёт BUSY; неверные размеры - BAD_ARGUMENT, формат - FORMAT.

Запрос: STREAM_CREATE_REQ (20 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	rate	u32	4	-
4	channels	u8	1	-
5	bits	u8	1	-
6	sample_format	u8	1	-
7	reserved	u8	1	-
8	buffer	u32	4	-
12	prefill	u32	4	-
16	low	u32	4	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

45h STREAM_STATE

Читает состояние своего кольца. fill/free - байты; published/consumed/position - PCM-кадры; fraction - дробная часть Q0.32. Сверяйте next_sequence после потери ответа FEED. После CLOSE объект исчезает и запрос даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK: STREAM_STATE (70 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	state	u8	1	-
3	flags	u8	1	-
4	buffer	u32	4	-
8	prefill	u32	4	-
12	low	u32	4	-
16	fill	u32	4	-
20	free	u32	4	-
24	published	u64	8	-
32	consumed	u64	8	-
40	next_sequence	u32	4	-
44	underruns	u32	4	-
48	rate	u32	4	-

Смещ.	Поле	Тип	Байт	Примечание
52	play_id	u16	2	-
54	channels	u8	1	-
55	bits	u8	1	-
56	error	u8	1	-
57	reserved	u8	1	-
58	fraction	u32	4	-
62	position	u64	8	-

46h STREAM_START

Однократно занимает основной звук и возвращает play_id. gain=0..64, pan=0..255. До prefill звучит тишина. Повторный START даёт STATE, нехватка голосов - NO_VOICE. Независимые FX сохраняются.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	gain	u8	1	-
3	pan	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-

47h STREAM_FEED

Отправляет 1..1024 байта полных PCM-кадров; первый sequence=1, transport=00h/80h. Между запросом и ответом нужно обслужить WRITE_GRANT. WRITE_END не нужен. Неверный sequence даёт RANGE, выравнивание - LENGTH, нехватка места в кольце - NO_MEMORY. Успех возвращает следующий sequence и published в кадрах.

Запрос: STREAM_FEED_REQ (9 байт FAST; 13 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	sequence	u32	4	-
6	length	u16	2	-
8	transport	u8	1	-
9	crc	u32	4	только VERIFIED

Ответ при OK: STREAM_FEED_RESP (12 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	next_sequence	u32	4	-
4	published	u64	8	-

48h STREAM_EOF

Сообщает, что новых данных не будет. Остаток доигрывается даже при fill меньше prefill. После этого FEED даёт STATE. Дождитесь state=3 и вызовите CLOSE: EOF сам по себе не освобождает кольцо.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

49h STREAM_CLOSE

Закрывает указанное кольцо и освобождает его после применения события звуковой частью. Состояние closing можно увидеть в STATE. Повтор во время закрытия даёт STATE, после удаления - HANDLE; заполненная очередь событий - BUSY.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

50h SNAPSHOT

Фиксирует согласованную сводку и записи голосов/FX. Для страниц используйте возвращённые session и snapshot_seq. Новый SNAPSHOT или SNAP_LATCH заменяет удерживаемую копию. Сам запрос не останавливает звук.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: SNAPSHOT (64 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	snapshot_seq	u32	4	-
4	session	u64	8	-
12	output_frames	u64	8	-
20	play_id	u16	2	-
22	reserved20	u16	2	значение 0
24	state	u8	1	-

Смещ.	Поле	Тип	Байт	Примечание
25	kind	u8	1	-
26	channels	u16	2	-
28	object	u16	2	-
30	reserved30	u16	2	значение 0
32	source_frame	u32	4	-
36	order	u16	2	-
38	row	u8	1	-
39	tick	u8	1	-
40	speed	u8	1	-
41	bpm	u8	1	-
42	peak_left	u16	2	-
44	peak_right	u16	2	-
46	audio_underruns	u32	4	-
50	last_audio_error	u8	1	-
51	reserved51	u8	1	значение 0
52	voice_count	u16	2	-
54	fx_count	u16	2	-
56	transfer_id	u32	4	-
60	transfer_done	u16	2	-
62	transfer_length	u16	2	-

Разрешена во время передачи блока.

51h FX_STATUS

Читает записи FX из удерживаемого снимка. first - индекс записи, max_records=1..6; большее значение ограничивается вместимостью ответа, ноль даёт BAD_ARGUMENT. Stereo занимает одну запись с channels=2. Неверный ключ снимка даёт STATE, first>total - RANGE; first=total возвращает count=0.

Запрос: PAGE_REQ (16 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	session	u64	8	-
8	snapshot_seq	u32	4	-
12	first	u16	2	-
14	max_records	u16	2	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	header	PAGE_HEADER	20	-
20	records	FXREC[]	по count	count записей указанного типа

Разрешена во время передачи блока.

52h VOICE_STATUS

Читает все физические голоса, включая незанятые. first - индекс записи, max_records=1..5; большее значение ограничивается вместимостью ответа, ноль даёт BAD_ARGUMENT. Ключ берётся из SNAPSHOT. Неверный ключ даёт STATE, first>total - RANGE; first=total возвращает пустую страницу.

Запрос: PAGE_REQ (16 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	session	u64	8	-
8	snapshot_seq	u32	4	-
12	first	u16	2	-
14	max_records	u16	2	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	header	PAGE_HEADER	20	-
20	records	VOICEREC[]	по count	count записей указанного типа

Разрешена во время передачи блока.

11.7. Вложенные структуры и записи

PCM_DESC (24 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	rate	u32	4	-
4	channels	u8	1	-
5	bits	u8	1	-
6	sample_format	u8	1	0=U8 1=S8 2=S16LE
7	flags	u8	1	Бит 0: петля вперёд
8	data_offset	u32	4	-
12	data_length	u32	4	-
16	loop_start	u32	4	-
20	loop_end	u32	4	-

PAGE_HEADER (20 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	session	u64	8	-
8	snapshot_seq	u32	4	-
12	first	u16	2	-
14	count	u16	2	-
16	total	u16	2	-
18	record_size	u16	2	-

VOICEREC (20 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	voice_id	u16	2	-
2	instance_id	u16	2	-
4	handle	u16	2	-
6	logical_channel	u16	2	-
8	source_frame	u32	4	-
12	peak	u16	2	-
14	period	u16	2	-
16	note	u8	1	-
17	volume	u8	1	-
18	pan	u8	1	-
19	flags	u8	1	-

FXREC (16 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	instance_id	u16	2	-
2	voice_count	u16	2	-
4	handle	u16	2	-
6	state	u8	1	0=неактивен, 1=играет, 2=пауза
7	channels	u8	1	-
8	source_frame	u32	4	-
12	gain	u8	1	-
13	pan	u8	1	-
14	priority	u8	1	-
15	flags	u8	1	Бит 0: подмена канала GS; бит 1: повтор всего объекта